SpiralTrain
developer courses



Python Programming

SpiralTrain developer courses

General Course Remarks

- Course times
- Drinks
- Meals
- Toilets
- Smoking
- Mobile Phones
- Manual
- Software

Everything should be made as simple as possible, but not simpler

Albert Einstein

Ja, was ich hier geschrieben habe, macht im Einzelnen überhaupt nicht den Anspruch auf Neuheit; und darum gebe ich auch keine Quellen an, weil es mir gleichgültig ist, ob das was ich gedacht habe, vor mir schon ein anderer gedacht hat.

Ludwig Wittgenstein

Python Intro

- Python Intro
- Variables and Data Types
- Data Structures
- Control Flow
- Functions
- Modules
- Comprehensions
- Exception Handling
- Python IO
- Python Database Access
- Python Classes
- APPENDIX Type Hinting
- APPENDIX Python Libraries

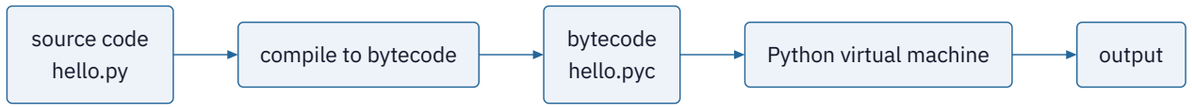
- What is Python?
- Python Features
- History of Python
- Modern Python Tooling
- Getting Started
- Setting up PATH
- Creating a Project
- Adding Dependencies
- Running Python
- Python in Interactive Mode
- Python in Script Mode
- Linting with ruff
- Editor Setup
- Python Identifiers
- Python Reserved Words

- Comments in Python
- Lines and Indentation
- Multi Line Statements
- Quotes in Python
- Summary : Python Intro

SpiralTrain developer courses

What is Python?

- **Python is a general purpose high-level scripting language :**
  - Supports many applications from text processing to WWW browsers and games
- **Python is Beginner's Language :**
  - Great language for the beginner programmers
- **Python is interpreted :**
  - Processed at runtime by the interpreter, with no separate compile step to run



- **Python is interactive :**
  - Prompt allows you to interact with the interpreter directly to write your programs
- **Python is Object Oriented :**
  - Supports Object-Oriented style programming that encapsulates code within objects
- **Python is easy to learn and easy to read :**
  - Python has relatively few keywords, simple structure, and a clearly defined syntax
  - Python code is much more clearly defined and visible to the eye

SpiralTrain developer courses

Python Features

- **Broad standard library :**
  - Files, paths, dates, JSON, CSV, regular expressions, SQLite and testing out of the box
- **Portable :**
  - The same code runs on Linux, Windows and macOS
- **Large ecosystem :**
  - Hundreds of thousands of packages on PyPI, installed with `uv add`
- **Used across many fields :**
  - Automation and scripting, web back ends, data analysis, machine learning, testing
- **Extendable :**
  - Performance-critical parts can be written in C or Rust and called from Python
- **Automatic memory management :**
  - Objects are freed by reference counting and a garbage collector, not by hand

SpiralTrain developer courses

History of Python

- **Python was developed by Guido van Rossum around 1990 at :**
  - National Research Institute for Mathematics and Computer Science in Netherlands
- **Python is derived from many other languages :**
  - Including Modula-3, C, C++, Smalltalk, Unix shell and other scripting languages
- **Python is usable as :**
  - Scripting language or compiled to byte-code for building large applications
- **Python is copyrighted and the source is open :**
  - Released under the PSF License, a permissive licence that allows commercial use
- **Python is developed in the open by a community of core developers :**
  - The Python Software Foundation holds the rights, the Steering Council decides
- **Python supports :**
  - Functional and structured programming methods as well as OOP
  - Very high-level dynamic data types and dynamic type checking

SpiralTrain developer courses

Modern Python Tooling

- **Python needs an interpreter, an environment and a set of packages :**
  - Historically each of these was installed and managed by a separate tool
  - The result was hard to reproduce on a second machine
- **Two tools now cover almost all of it :**
  - `uv` installs Python, creates the environment and resolves the packages
  - `ruff` checks style and catches a large class of mistakes before you run
- **Both are single binaries with no Python of their own to bootstrap :**

Older tool	Replaced by
`python.org` installer, `pyenv`	`uv python install`
`venv` , `virtualenv`	`uv` creates `.venv` for you
`pip` , `pip-tools` , `requirements.txt`	`uv add` and `uv.lock`
`flake8` , `isort` , `black` , `pylint`	`ruff check` and `ruff format`

SpiralTrain developer courses

Getting Started

- **Install `uv` once, and let it install Python for you :**

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

- **On macOS or Linux use the shell installer :**

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

- **Close and reopen the terminal, then fetch the interpreter :**

```
uv python install 3.14
```

- **This needs no administrator rights :**
  - Any Python already on the machine is left alone, and versions live side by side
- **The traditional route is the installer from `https://www.python.org/` :**
  - Tick `Add python.exe to PATH` on the first page of the Windows wizard
  - Python 2 reached its end of life in 2020, so Python 3 is the only choice
- **Documentation lives at `https://docs.python.org/3/`**

SpiralTrain developer courses

Setting up PATH

- **Only needed when Python was installed by hand :**
  - `uv` puts the interpreter where `uv run` can find it, so it needs none of this
- **PATH is an environment variable :**
  - Provides a search path that lists directories that OS searches for executables
  - Named PATH in Unix or Path in Windows, and Unix is case-sensitive
- **Setting PATH in Linux :**

```
export PATH="$PATH:/usr/local/bin/python"
```

- **Setting PATH in Windows :**

```
PATH=%PATH%;C:\Python
```

- **Or use the environment variables tab under System Properties**
- **PYTHONPATH is a separate variable :**
  - It tells the interpreter where to find modules you import, not where Python is

SpiralTrain developer courses

Creating a Project

- **A project is a directory with a `pyproject.toml` in it :**

```
uv init course-work
cd course-work
```

- **`uv init` writes three things :**
  - `pyproject.toml` , the file that names the project and its dependencies
  - `.python-version` , the interpreter this project is pinned to
  - `main.py` , a placeholder to delete or edit

```
course-work/
├─ pyproject.toml
├─ .python-version
├─ main.py
└─ .venv/              created on the first uv run or uv add
```

- **Run code through `uv run` , which uses the project's own interpreter :**

```
uv run python main.py
```

Adding Dependencies

- `uv add` **installs a package and records it in** `pyproject.toml` **:**

```
uv add requests
uv add "requests>=2.28,<3"
uv add --dev pytest
```

- **Two files describe the dependencies, and both belong in version control :**
  - `pyproject.toml` states what the project asks for
  - `uv.lock` states the exact versions that were resolved
- **Anyone can then reproduce the environment exactly :**

```
uv sync          # install exactly what uv.lock says
uv remove requests
```

- `--dev` **marks a tool used while developing, not by the running program :**
  - `ruff` is such a tool; so is the test runner `pytest`, which the libraries appendix covers
- **There is no environment to activate :**
  - `uv run` puts the project's `.venv` in front for the length of one command

SpiralTrain developer courses

Running Python

- **Inside a project, `uv run python` starts the project's own interpreter :**
  - A plain `python` works too, where a Python is on the PATH
- **Interactive Interpreter :**
  - Start the interactive interpreter by entering `uv run python` with no file
  - The interpreter answers with its `>>>` prompt, ready for a statement
  - `python -h` lists the command line options
- **Script from Command Line :**
  - A Python script is executed by passing its file name to the interpreter

```
uv run python script.py
```

- **Integrated Development Environment :**
  - Run and debug Python from an editor, which is how most work is done
  - This course uses VS Code, set up two slides on
- **Jupyter Notebook on a web server :**
  - Cell by cell execution, common in data science work

SpiralTrain developer courses

Python in Interactive Mode

- **Start with Hello World Python program in interactive mode :**
  - Invoke the interpreter by typing `uv run python` in a terminal

```
C:\course-work> uv run python
Python 3.14.0 (main, Oct  7 2025, 10:12:33) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello, Python!")
Hello, Python!
>>>
```

- **Type the statement right of the `>>>` prompt and press Enter :**
  - The interpreter evaluates it and prints the result immediately
- **Separate multiple statements with a semicolon ;**

SpiralTrain developer courses

Python in Script Mode

- **When the interpreter is invoked with a script parameter :**
  - The script begins execution and continues until it is finished
  - When the script is finished, the interpreter is no longer active
- **All Python files will have extension `.py` :**
  - Put the following source code in a `hello.py` file

```
print("Hello, Python!")
```

- **Run the script as follows :**

```
uv run python hello.py
```

- **This will produce following result :**

```
Hello, Python!
```

- **`print()` default prints a new line**

SpiralTrain developer courses

Linting with ruff

- **A linter reads code without running it and reports what looks wrong :**
  - Unused imports and variables, shadowed built-ins, unreachable code
  - Style that departs from the conventions the rest of Python follows
- **Two commands cover the daily use :**

```
uv run ruff check .           # report problems
uv run ruff check . --fix     # fix the ones that are safe to fix
uv run ruff format .          # lay the code out consistently
```

- **Rules are configured in `ruff.toml`, which ships with this course :**
  - It selects the rule set the course material is written against
  - Without it the editor would flag patterns the course uses on purpose

demo03 · ruff_findings

SpiralTrain developer courses

Editor Setup

- **The course demonstrates Visual Studio Code :**
  - Any editor works, but the slides assume this one
- **Install two extensions from the Extensions panel :**
  - `Python` , published by Microsoft
  - `Ruff` , published by Astral
- **With both installed the editor will :**
  - Underline the problems `ruff check` would report, as you type
  - Offer the same fixes, and format on save if you ask it to
  - Resolve imports against the project's `.venv` rather than a global Python
- **Point the editor at the project interpreter :**
  - Command Palette, `Python: Select Interpreter` , choose the one in `.venv`

SpiralTrain developer courses

Python Identifiers

- **Identifiers are the names of variables, functions, classes and modules :**
  - Start with a letter or an underscore, followed by letters, digits and underscores
  - Punctuation such as `@` , `$` and `%` is not allowed in a name
- **Python is case sensitive :**
  - `total` and `Total` are two different names
- **The naming conventions of PEP 8, the Python style guide :**
  - `snake_case` for variables, functions and modules
  - `CapWords` for classes, `UPPER_CASE` for values meant as constants
- **Underscores carry meaning :**
  - A leading underscore, as in `_cache` , marks a name as internal to its module
  - Names such as `__init__` with two underscores on both sides are defined by Python

SpiralTrain developer courses

Python Reserved Words

- **Keywords may not be used as a variable, function or other name :**

```
False    None     True     and      as       assert   async
await    break    class    continue def      del      elif
else     except   finally  for      from     global   if
import   in       is       lambda   nonlocal not      or
pass     raise    return   try      while    with     yield
```

- `match`, `case`, `type` **and** `_` **are soft keywords :**
  - Reserved only inside the statement they belong to, and usable as names elsewhere
- **The interpreter can list them :** `import keyword` **and print** `keyword.kwlist`

SpiralTrain developer courses

Comments in Python

- `#` **starts a comment outside a string :**
  - Python ignores the rest of that line
  - A comment explains a reason or a constraint

```
# Keep the original filename so the photo can be found again.
filename = "IMG_0421.jpg"
print(filename)  # Show the selected file.
```

- **Several comment lines each start with** `#` **:**

```
# Titles are supplied by the playlist file.
# The music files themselves are kept in a separate folder.
```

- **Text inside quotes is part of the string :** `"Track #1"` **contains a hash sign**

Lines and Indentation

- **Indentation groups statements into a block :**
  - Use four spaces for each level
  - Statements in the same block have the same indentation

```
if True:  # The indented block runs.
    print("Photo ready")
else:
    print("Photo missing")
```

- **An unindented statement runs after the block :**

```
photos = 5
# This comment does not change the photo count.
print("Photos selected:", photos)
```

- **Inconsistent indentation raises an error before the script runs**
- `end=" "` **makes the next output continue on the same line**

Blocks and Line Endings

```
if True:  # The indented block runs.
    print("Photo ready")
else:
    print("Photo missing")

photos = 5
# This comment does not change the photo count.
print("Photos selected:", photos)
print("Playing", end=" ")
print("Imagine")
```

OUTPUT

```
Photo ready
Photos selected: 5
Playing Imagine
```

WRITTEN BY PRINT()

Photo ready newline

Photos selected: 5 newline

Playing Imagine newline

The next `print()` continues on the same line, then ends it

demo04 · lines_and_identation

SpiralTrain developer courses

Multi Line Statements

- **Parentheses allow a calculation to continue across lines :**

```
thriller = 357
imagine = 183
yesterday = 125

total = (
    thriller + imagine + yesterday
)
print(total)  # 665
```

- **Lists, dictionaries and calls can also span lines inside their brackets :**

```
playlist = [
    "Thriller", "Imagine", "Yesterday",
]
print(playlist)
```

- **A backslash also continues a line :**
  - Prefer brackets when they fit the statement
  - A semicolon separates statements on one line; separate lines are clearer

demo05 · multilinestatements

SpiralTrain developer courses

Quotes in Python

- **Matching quotes mark the start and end of a string :**
  - Single and double quotes produce the same string type
  - Triple quotes allow the text to span lines

```
title = "Imagine"
filename = 'cover.jpg'
caption = """Friday night
Live at the observatory"""
```

- **Choose double quotes when the text contains an apostrophe :**

```
print("Tonight's playlist")
print('Tonight\'s playlist')
```

demo06 · quotesandescapes

SpiralTrain developer courses

Summary : Python Intro

- **Python is a general purpose high-level scripting language :**
  - Interpreted, interactive and object oriented, developed by Guido van Rossum
- **`uv` manages the interpreter, the environment and the packages :**
  - `uv python install`, `uv init`, `uv add`, `uv run`
  - `pyproject.toml` states what is wanted, `uv.lock` pins what was resolved
- **`ruff` reports style and correctness problems without running the code**
- **Python can be started in three different ways :**
  - Interactive interpreter, script from the command line, from inside an IDE
- **Python is case sensitive, and names follow PEP 8 :**
  - `snake_case` for variables and functions, `CapWords` for classes
- **Python uses line indentation to denote blocks of code :**
  - No braces for class and function definitions or for flow control
- **The line continuation character () denotes that a line should continue**
- **Quotes denote string literals, single, double or triple**

Variables and Data Types

Python Intro

Variables and Data Types

Data Structures

Control Flow

Functions

Modules

Comprehensions

Exception Handling

Python IO

Python Database Access

Python Classes

APPENDIX Type Hinting

APPENDIX Python Libraries

Variables

Everything is an Object

Multiple Assignment

Strong Typing

Annotating Variables

Python Numbers

Type Conversions

Conversion Functions

Built-in Number Functions

Python Operators

Arithmetic and Assignment Operators

Comparison Operators

Logical, Membership, Identity Operators

String Operators

String Operations

Escape Characters

Raw and Unicode Strings

f-Strings

Format Specifications

Triple Quotes

Built-in String Methods

Cleaning Photo Filenames

Summary : Variables and Data Types

SpiralTrain developer courses

Variables

- **Variables are names bound to objects :**
  - Assignment binds a name to the object produced by an expression
  - Several names can refer to the same object
- **Objects have types, such as `int`, `float` and `str` :**
  - `type(x)` returns the type of the object currently bound to `x`
  - Rebinding `x` can make it refer to an object of a different type
- **`id(x)` identifies the object while it exists**
- **The `=` operator binds the name on its left to the object on its right :**
  - A name does not need a separate declaration before assignment

```
duration = 222  # Track length in seconds
size_mb = 8.5  # Photo file size
title = "Imagine"  # Track title
```

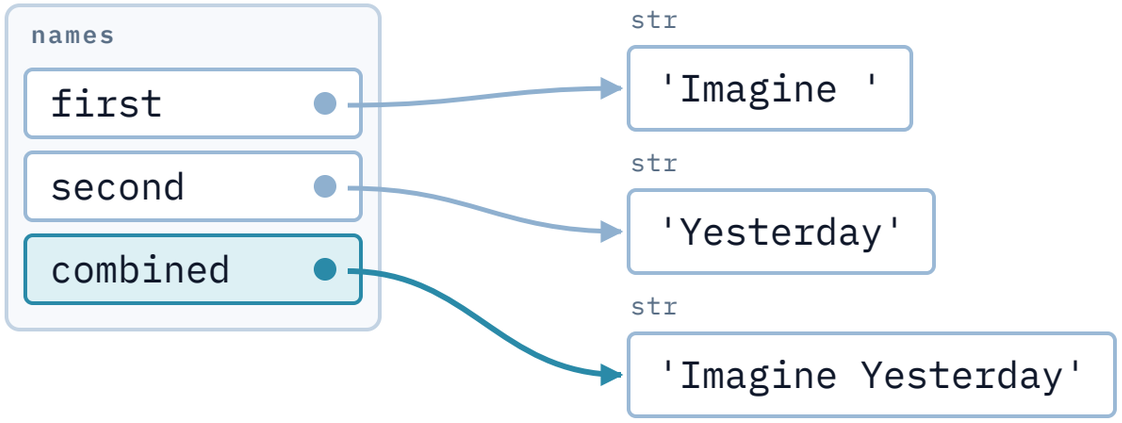
- **Uppercase names such as `MAX_VOLUME` mark values intended to stay constant**

Binding and Rebinding Names

```
first = 10
second = 20
combined = first + second
print(combined)
first = "Imagine "
second = "Yesterday"
combined = first + second
print(combined)
```

OUTPUT

```
30
Imagine Yesterday
```

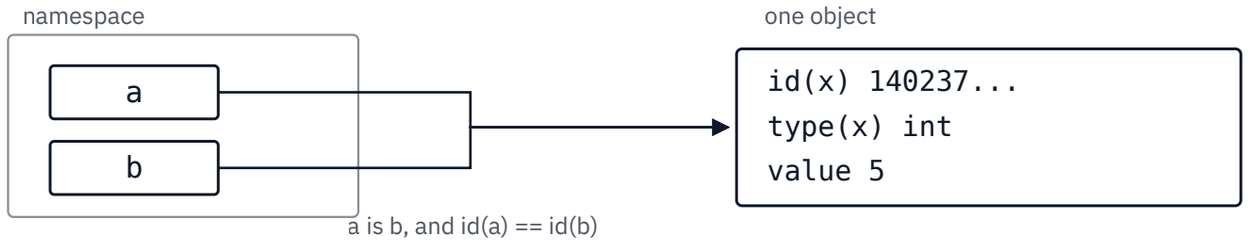


The same names and the same code, now working on strings

SpiralTrain developer courses

Everything is an Object

- **Each object `x` has :**
  - Unique identifier, an integer, returned by `id(x);`
  - Type returned by `type(x)`
  - Value may or may not be changeable :
    - Value can itself consist of one or more objects
  - Zero or more methods provided by type object to get and/or change content
  - Zero or more names
- **Objects have individuality :**
  - Multiple names in multiple scopes can be bound to the same object
  - This is known as aliasing
- **Variable names are not properties of an object :**
  - A namespace maps each name to the object it refers to



demo02 · typeandid

SpiralTrain developer courses

Multiple Assignment

- **Simultaneous assignment of values :**

```
a = b = c = 1
```

- **All three names are bound to the same integer object**
- **Reassigning one name changes only that binding :**
  - The other names still refer to the original object
- **Integers are immutable :**
  - Rebinding a name does not change the integer it previously named
  - The new binding may refer to an already existing object
- **Assignment of multiple objects to multiple variables :**

```
a, b, c = 1, 2, "Imagine"
```

- **Two integer objects with values 1 and 2 :**
  - Assigned to variables a and b
- **One string object with the value `"Imagine"` :**
  - Assigned to the variable c

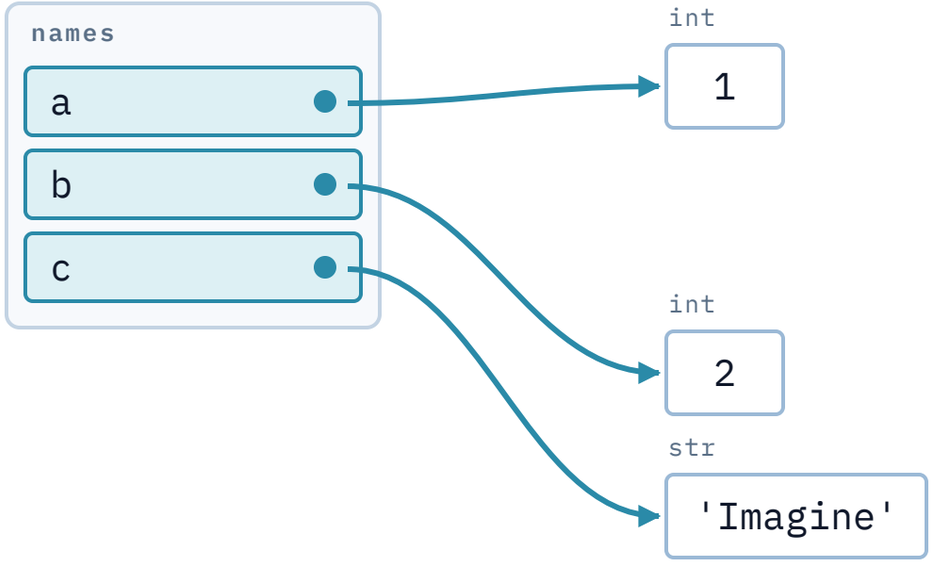
demo03 · multipleassignment

Several Names for One Object

```
a = b = c = 1
print(a, b, c)
a = 11
print(a, b, c)
a, b, c = 1, 2, "Imagine"
print(a, b, c)
```

OUTPUT

```
1 1 1
11 1 1
1 2 Imagine
```



The three names now refer to three different objects

SpiralTrain developer courses

Strong Typing

- **An object's type determines which operations it supports :**
  - A duration stored as a number can be added to another duration
  - A duration read as text must be converted before doing arithmetic

```
222 + 30        -> 252          integer addition
"222" + "30"    -> '22230'      string concatenation
"222" + 30      -> TypeError    string and integer do not add
```

- **Python reports an unsupported combination instead of silently converting it**
- **Common types include numbers, strings, lists, tuples, sets and dictionaries**

demo04 · strongtyping

Annotating Variables

- **A variable annotation records what a value is intended to represent :**

```
seconds: int = 222
size_mb: float = 8.5
title: str = "Imagine"
is_favourite: bool = True
```

- **An annotation does not change runtime behaviour :**
  - It does not convert a value or prevent another type from being assigned
  - Editors and type checkers use it to find mismatches
- **An annotation is useful when the intended type is unclear :**
  - `total_mb: float = 0` records that a total may later include fractions
- **Collection annotations and function signatures are introduced with those subjects**

SpiralTrain developer courses

Python Numbers

- **Number objects are immutable; assignment changes a name's binding :**
  - `previous` keeps the original duration when `duration` is rebound

```
duration = 222
previous = duration

duration = 185
print(previous, duration)  # 222 185

del previous
print(duration)  # 185
```

- `del` **removes a name; it does not change the number**
- **Other references keep their objects available**

SpiralTrain developer courses

Type Conversions

- Convert text explicitly when it needs to represent a number :

```
seconds_text = "222"
seconds = int(seconds_text)
print(seconds + 30)  # 252
print(float("8.5"))  # 8.5
print(int(8.9))  # 8, the fractional part is discarded
```

- `bool` converts a value using its truth value :

```
print(bool("Imagine"))  # True
print(bool(""))  # False
print(bool(222))  # True
print(bool(0))  # False
```

- Nonzero numbers and nonempty strings are truthy
- Zero, empty strings, empty collections and `None` are falsy
- `int("ff", 16)` reads a number written in hexadecimal

demo05 · numbertypeconversion

Conversion Functions

- Convert text to a number before using it in arithmetic :

```
duration_text = "222"
duration = int(duration_text)
print(duration + 30)  # 252
print(str(duration))  # 222
print(repr(" Imagine "))  # ' Imagine '
```

- `str` produces display text; `repr` exposes quotes and whitespace
- Other conversions are useful when inspecting character and byte values :

Function	Result
`chr(65)`	`'A'`
`ord("A")`	`65`
`hex(65)`	`'0x41'`
`oct(65)`	`'0o101'`

demo06 · conversionfunctions

Built-in Number Functions

Function	Description
`abs(x)`	Absolute value of x: the (positive) distance between x and zero
`ceil(x)`	The ceiling of x: the smallest integer greater than or equal to x
`exp(x)`	The exponential of x: e**x
`fabs(x)`	The absolute value of x, is part of math module
`floor(x)`	The floor of x: the largest integer not greater than x
`log(x)`	The natural logarithm of x, for x > 0
`log10(x)`	The base-10 logarithm of x, for x > 0
`pow(x, y)`	The value of x**y
`sqrt(x)`	The square root of x, for x >= 0
`max(x, y, ..)` / `min(x, y, ..)`	The maximum or minimum of a list of numbers

- **For some of these functions the `math` module must be imported**

`demo07 · numberfunctions`

SpiralTrain developer courses

Python Operators

- **An operator combines one or more operands into a new value :**

```
print(7 / 2)  # 3.5, true division always gives a float
print(7 // 2)  # 3, floor division rounds down
print(7 % 2)  # 1, the remainder
print(2**10)  # 1024, exponentiation
```

- **Python groups its operators as follows :**
  - Arithmetic and assignment operators, for numbers and for strings
  - Comparison operators, which give `True` or `False`
  - Logical, membership and identity operators
- **Bitwise operators `& | ^ ~ << >>` work on the bits of integers, rarely needed**
- **There is no `++` or `--` : write `count += 1` instead**

Arithmetic and Assignment Operators

Operator	Arithmetic Operator Description
`+` / `-`	Adds or subtracts the values on either side of the operator
`*`	Multiplies the values on either side of the operator
`/`	Divides left operand by right operand, the result is always a float
`//`	Floor division, the quotient rounded down to a whole number
`%`	Modulo, the remainder of the division
`**`	Exponentiation, the left operand raised to the power of the right

Operator	Assignment Operator Description
`=`	Binds the name on the left to the value on the right
`+=` / `-=`	Adds or subtracts the right operand, and assigns the result to the left
`*=` / `/=`	Multiplies or divides by the right operand, and assigns the result
`//=` / `%=` / `**=`	The same for floor division, modulo and exponentiation

```
count = 10
count += 1
count *= 2
print(count)  # 22
```

SpiralTrain developer courses

Comparison Operators

Operator	Comparison Operator Description
`==` / `!=`	True if the two values are equal, or not equal
`>` / `<`	True if the left value is greater, or less, than the right
`>=` / `<=`	True if the left value is greater or equal, or less or equal

- **Comparisons chain, so a range check reads as it would on paper :**

```
i = 10
j = 11
print(i > j)  # False
print(i != j)  # True
print(0 < i < 20)  # True, comparisons chain
print("apple" < "banana")  # True, strings compare letter by letter
```

- **`==` compares values; whether two names are one object is `is`, on the next slide**

Logical, Membership, Identity Operators

Operator	Description
`and` / `or`	True if both operands are true, or if at least one is
`not`	Reverses the truth value of its operand
`in` / `not in`	True if the value is, or is not, an element of the sequence
`is` / `is not`	True if both sides are, or are not, the very same object

- **`and` and `or` stop as soon as the answer is known, and return an operand :**

```
name = ""
print(name or "anonymous")  # anonymous, `or` returns the first truthy operand
print(name and "anonymous")  # an empty line, `and` returns the first falsy operand
```

- **`x if c else y` gives `x` when `c` is true, and `y` when it is false :**

```
age = 17
print("adult" if age >= 18 else "minor")  # minor
```

- **`is` asks whether two names refer to one object, `==` whether values are equal :**

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]
print(a == c)  # True, equal values
print(a is c)  # False, two different lists
print(a is b)  # True, two names for one list
```

SpiralTrain developer courses

Python Operators Precedence

Operator	Description, from highest to lowest
`**`	Exponentiation, which groups from the right
`+x` `-x` `~x`	Unary plus, minus and bitwise not
`*` `/` `//` `%`	Multiplication, division, floor division and modulo
`+` `-`	Addition and subtraction
`<<` `>>` `&` `^` `|`	Bitwise shifts, and, exclusive or, or
`==` `!=` `<` `<=` `>` `>=` `in` `not in` `is` `is not`	Comparisons, membership and identity
`not`	Logical not
`and`	Logical and
`or`	Logical or

```
print(2 + 3 * 4)  # 14
print((2 + 3) * 4)  # 20
print(2**3**2)  # 512, ** groups from the right
```

- **When in doubt, add parentheses : they cost nothing and help the reader**

SpiralTrain developer courses

Python Strings

- **A string is a sequence of Unicode characters between matching quotes :**
  - Single and double quotes produce the same type
  - Triple quotes allow the text to span lines

```
filename = "IMG_0421.jpg"
title = "Imagine"
```

- **Indexing selects one character; slicing selects a section :**
  - The first position is `0`; negative indexes count from the end
  - The stop position of a slice is excluded
  - A single character is still a string
- **Strings are immutable :**
  - A changed filename is a new string; the original value is preserved

String Operators

Operator	Description
`+`	Concatenation. Adds values on either side of the operator
`*`	Repetition. Creates string by concatenating multiple copies of same string
`[]`	Slice. Gives the character from the given index
`[:]`	Range Slice. Gives the characters from the given range
`in`	Membership. Returns true if a character exists in the given string
`not in`	Membership. Returns true if a character does not exist in the given string
`r/R`	Raw String. Suppresses actual meaning of escape characters

SpiralTrain developer courses

String Operations

- Indexes and slices select parts of a photo filename :

```
filename = "IMG_0421.jpg"
print(filename[0])  # I
print(filename[-1])  # g
print(filename[4:8])  # 0421
print(filename[-4:])  # .jpg
print(".jpg" in filename)  # True
```

- Concatenation builds a new name without changing the original :

```
edited = filename[:-4] + "_edit.jpg"
print(edited)  # IMG_0421_edit.jpg
print(filename)  # IMG_0421.jpg
```

- Repetition can produce a separator : `"-" * 12`
- A negative step reverses the characters : `filename[::-1]`

SpiralTrain developer courses

Escape Characters

- **Escape or non-printable characters :**
  - Characters that can be represented with backslash notation

Backslash Notation	Hexadecimal Character	Description
`\`	0x5c	Backslash
`\'` / `\"`	0x27 / 0x22	Single or double quote
`\n`	0x0a	Newline
`\r`	0x0d	Carriage return
`\t`	0x09	Tab
`\a`	0x07	Bell or alert
`\b`	0x08	Backspace
`\ooo`		Character with octal value ooo, digits 0 to 7
`\xhh`		Character with hexadecimal value hh
`\uxxxx`		Unicode character with 16-bit hexadecimal value xxxx

`demo11 · escapecharacters`

SpiralTrain developer courses

Raw and Unicode Strings

- **Raw String don’t treat backslash as a special character :**
  - Every character in a raw string stays the way it was
- **In a normal String the backslash can be escaped :**

```
print("C:\\nowhere")
```

- **Result :** `C:\nowhere`
- **A raw string is made with the** `r` **in front of the String :**

```
print(r"C:\nowhere")
```

- **Result :** `C:\nowhere`
- **Python strings are Unicode :**
  - A string is a sequence of code points, so `len("Ä…Ä‡")` is 2
  - Text in almost every script in the world can be stored and processed
- `unicodedata` **module** `:`
  - Provides access to Unicode Character Database (UCD)
  - UCD contains character properties for all Unicode characters

f-Strings

- **An f-string inserts values into text using `{...}` fields :**
  - Put `f` before the opening quote
  - Fields can contain arithmetic and method calls

```
title = "Imagine"
seconds = 222
print(f"Now playing: {title.upper()}")
print(f"With a 30-second intro: {seconds + 30} seconds")
```

```
Now playing: IMAGINE
With a 30-second intro: 252 seconds
```

- **`!r` shows quotes and whitespace; `=` also shows the field's name :**

```
print(f"{title!r}")  # 'Imagine', quotes remain visible
print(f"{seconds = }")  # seconds = 222, useful during debugging
```

SpiralTrain developer courses

Format Specifications

- A colon inside a field introduces its display format :

```
photo_id = 421
size_mb = 8.537
print(f"Photo {photo_id:05d}: {size_mb:6.2f} MB")
```

- The result is `Photo 00421:  8.54 MB`

Format	Effect
`05d`	Integer padded to five digits with zeroes
`6.2f`	Number six characters wide, with two decimal places
`<10` , `^10` , `>10`	Left, centre or right alignment in ten characters
`:,.0f`	Thousands separators and no decimals
`:.0%`	Percentage with no decimals

- Older code uses `%` or `str.format` ; recognise these forms :

```
print("Photo %05d: %6.2f MB" % (photo_id, size_mb))
print("Photo {:05d}: {:6.2f} MB".format(photo_id, size_mb))
```

demo13 · formatspecs

SpiralTrain developer courses

Triple Quotes

- A triple-quoted string keeps line breaks written in the source :

```
message = """Tonight's playlist
1. Imagine
2. Yesterday
Bring headphones."""
print(message)
```

```
Tonight's playlist
1. Imagine
2. Yesterday
Bring headphones.
```

- Both triple single quotes and triple double quotes are supported
- Escape sequences such as `\n` and `\t` still work inside the string

demo14 · triplequotes

SpiralTrain developer courses

Built-in String Methods

- **String methods are called with dot notation :** `"Imagine".upper()`
  - A string never changes, so each method returns a new string or a value

Method	Description
`strip()`	Removes whitespace from both ends, `lstrip()` and `rstrip()` from one
`lower()` / `upper()`	Converts the string to all lowercase or all uppercase
`split(sep)`	Splits the string into a list of words, on whitespace by default
`sep.join(words)`	Joins a list of strings into one, with `sep` between them
`count(sub)`	Returns how many times `sub` occurs in the string
`startswith(prefix)`	Returns `True` if the string starts with `prefix` , see also `endswith()`
`isalpha()`	Returns `True` if the string is not empty and holds only letters
`replace(old, new)`	Returns a copy with every `old` replaced by `new`
`find(sub)`	Returns the index of the first `sub` , or `-1` if it is not there
`isascii()`	Returns `True` if every character is plain ASCII, so `False` for a letter such as `é`
`splitlines()`	Splits the string at line breaks, one item per line, without the line breaks

- `len(s)` **is a function rather than a method, and gives the number of characters**
- **All the others :** `https://docs.python.org/3/library/stdtypes.html#string-methods`

SpiralTrain developer courses

Cleaning Photo Filenames

- A pasted filename can contain spaces and inconsistent capitalisation :
  - `strip` removes surrounding whitespace
  - `lower` produces a consistent name for comparisons

```
filename = "  IMG_0421.JPG  "
clean = filename.strip().lower()
print(repr(clean))  # 'img_0421.jpg'
print(clean.endswith(".jpg"))  # True
print(clean.replace(".jpg", "_edit.jpg"))  # img_0421_edit.jpg
```

- Each string method returns a result; `filename` still holds the original text
- Chained calls apply left to right, passing each result to the next method

demo15 · stringfunctions

SpiralTrain developer courses

Summary : Variables and Data Types

- **A variable is a name bound to an object :**
  - Created on first assignment with `=`, and can be annotated, as in `seconds: int = 222`
  - Every object has an identity, `id(x)`, a type, `type(x)`, and a value
- **Python is strongly typed :**
  - Converting between types is explicit, with `int()`, `float()`, `str()` and `bool()`
- **Numbers are `int`, `float` and `complex`; `bool` is a subtype of `int`**
- **Operators compute, compare and combine values :**
  - `==` compares values, `is` compares identities
- **Strings are immutable sequences of Unicode characters :**
  - Indexed and sliced with `[]`, and have methods such as `strip()` and `split()`
- **f-strings are the way to format text :**
  - `f"{size_mb:6.2f}"` sets the width and the decimals of the value it shows

Data Structures

Python Intro

Variables and Data Types

Data Structures

Control Flow

Functions

Modules

Comprehensions

Exception Handling

Python IO

Python Database Access

Python Classes

APPENDIX Type Hinting

APPENDIX Python Libraries

Sequences

Lists

Accessing Lists

Delete List Elements

Multidimensional Lists

List Operations

List Functions and Methods

Tuples

Accessing Tuples

Delete Tuple Elements

Usage of Tuples

Unpacking in for Loops

Tuple Functions

Sets

Set Operations

Dictionaries

Accessing Values in Dictionary

Properties of Dictionary Keys

Annotating Collections

Dictionary Functions and Methods

Summary : Data Structures

SpiralTrain developer courses

Sequences

- **Most basic data structure in Python is the sequence :**
  - Each element of a sequence is assigned a number, its position or index
  - First index is zero, the second index is one, and so forth
  - Python distinguishes sequences on the basis of mutability
- **Immutable sequences :**
  - Contain objects and content cannot change once created :
    - Objects may reference other objects, and those other objects may change
  - Examples are strings, tuples and bytes, which the IO module covers
- **Mutable sequences :**
  - Contain objects and content can be changed once created
  - The list is the example used everywhere
- **Operations that work on all sequences types :**
  - Indexing, slicing, multiplying, and checking for membership
- **Python has built-in functions for :**
  - Finding the length of a sequence and its largest and smallest elements

SpiralTrain developer courses

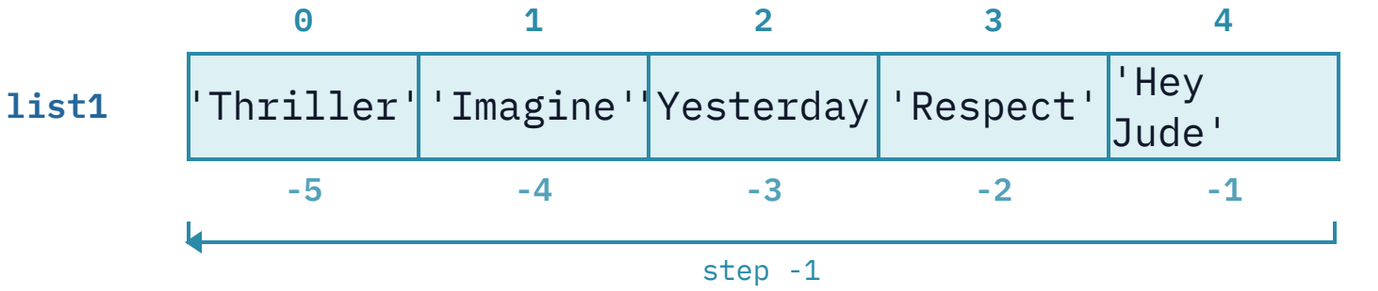
Lists

- **A playlist stores tracks in a chosen order :**
  - Lists are mutable sequences
  - List contains items separated by commas and enclosed within square brackets `[]`
  - The items of one list can be of different data types
- **Values are accessed using the index or slice operator `[ ]` and `[ : ]` :**
  - Indexes start at 0, then upwards and with end-1
  - Lists can be sliced, concatenated and so on
- **Lists respond to the `+` and * operators much like strings :**
  - Meaning is concatenation and repetition
  - Except that result is a new list, not a string
- **Lists respond to all sequence operations usable on strings :**

```
list1 = ["Thriller", "Imagine", "Yesterday", "Respect", "Hey Jude"]
print(list1)  # Prints complete list
print(list1[0])  # Prints first element of the list
print(list1[1:3])  # Prints the 2nd and 3rd elements
print(list1 * 2)  # Prints list two times
print(list1[::-1])  # Prints complete list in reverse order
```

Indexing and Slicing a List

```
list1 = ["Thriller", "Imagine", "Yesterday", "Respect", "Hey Jude"]
print(list1[0])  # Prints first element of the list
print(list1[1:3])  # Prints the 2nd and 3rd elements
print(list1[::-1])  # Prints complete list in reverse order
```



OUTPUT

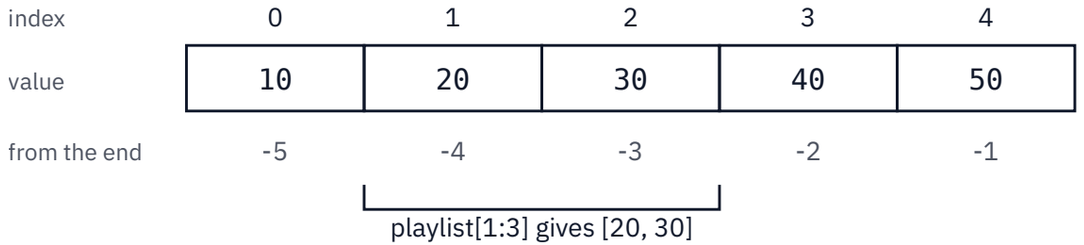
```
Thriller
['Imagine', 'Yesterday']
['Hey Jude', 'Respect',
'Yesterday', 'Imagine', 'Thriller']
```



A step of `-1` copies the items from the back to the front

Accessing Lists

- **Use index operator `[]` to access list at specific position :**
  - Counting runs from `0` at the front and from `-1` at the back
  - A slice runs up to but not including its second index




```
playlist = ["Thriller", "Imagine", "Yesterday", "Hey Jude"]
print("Value available at index 2 : ")
print(playlist[2])
playlist[2] = "Jolene"
print("New value available at index 2 : ")
print(playlist[2])
```

- **Elements can be added to a list with `append()` :**

```
list2 = [1, 2, 3]
list2.append(4)
list2.append(5)
print(list2)  # [1, 2, 3, 4, 5]
```

- **Elements can be added to a list at a position with `insert(index, item)` :**

```
list2.insert(0, 99)
print(list2)  # [99, 1, 2, 3, 4, 5]
```

Updating Lists

02_updatinglists.py

```
1  li2 = [1, 2, 3]
2  li2.append(4)
3  li2.append(5)
4  li2.insert(0, 99)
5  li2.extend([33, 44, 55])
```



demo02 · updatinglists

SpiralTrain developer courses

Delete List Elements

- Remove a list element with `del` or `remove()`
- Use `del` statement if you know index of element to remove :

```
playlist = ["Thriller", "Imagine", "Yesterday", "Hey Jude"]
del playlist[2]
print("After deleting value at index 2 : ")
print(playlist)  # ['Thriller', 'Imagine', 'Hey Jude']
```

- Use `remove()` if want to remove element by value :

```
playlist.remove("Thriller")
print("After deleting value 'Thriller' ")
print(playlist)  # ['Imagine', 'Hey Jude']
```

- List itself can also be removed with `del` :

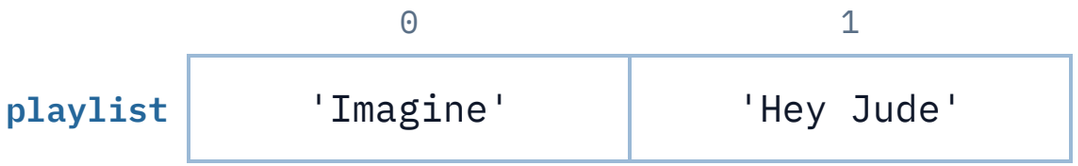
```
del list1
list1.append(100)  # gives NameError, name list1 not defined anymore
```

Replacing and Deleting Items

```
playlist = ["Thriller", "Imagine", "Yesterday", "Hey Jude"]
playlist[2] = "Jolene"
del playlist[2]
print(playlist)  # ['Thriller', 'Imagine', 'Hey Jude']
playlist.remove("Thriller")
print(playlist)  # ['Imagine', 'Hey Jude']
```

OUTPUT

```
['Thriller', 'Imagine', 'Hey Jude']
['Imagine', 'Hey Jude']
```



Both deletions changed the same list object in place

Multidimensional Lists

- **A tiny image can store brightness values in rows and columns :**
  - Each row is a list inside the outer list

```
grid = [
    [1, 2, 3],
    [4, 5, 6],
]
print(grid[0])  # [1, 2, 3], the first row
print(grid[1][2])  # 6, row 1, column 2
print(len(grid), len(grid[0]))  # 2 3, rows and columns
```

- **Each row is an ordinary list, so it can be changed in place or added to :**

```
grid[0][0] = 10
print(grid)  # [[10, 2, 3], [4, 5, 6]]
grid.append([7, 8, 9])
print(grid)  # [[10, 2, 3], [4, 5, 6], [7, 8, 9]]
```

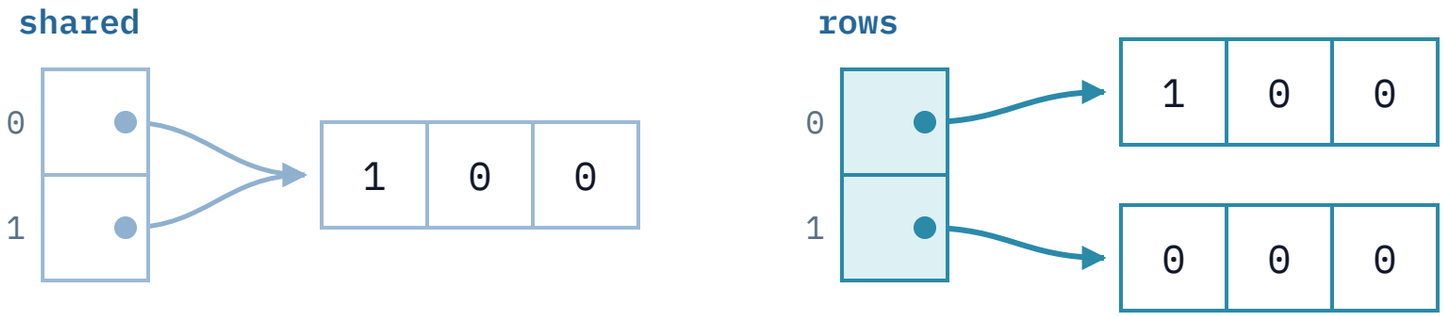
- **Repeating a row with `*` repeats one list object, not a copy of it :**

```
shared = [[0] * 3] * 2
shared[0][0] = 1
print(shared)  # [[1, 0, 0], [1, 0, 0]], both rows are one list
```

demo03 · multidimensionallists

Shared and Separate Rows

```
shared = [[0] * 3] * 2
shared[0][0] = 1
print(shared)  # [[1, 0, 0], [1, 0, 0]], both rows are one list
rows = [[0] * 3, [0] * 3]
rows[0][0] = 1
print(rows)  # [[1, 0, 0], [0, 0, 0]], two separate lists
```



OUTPUT

```
[[1, 0, 0], [1, 0, 0]]
[[1, 0, 0], [0, 0, 0]]
```

Each row is its own list, so row 1 is unchanged

List Operations

Python Expression	Results	Description
`len([1,2,3])`	`3`	Length
`[1,2,3]+[4,5,6]`	`[1,2,3,4,5,6]`	Concatenation
`['go']*3`	`['go','go','go']`	Repetition
`3 in [1,2,3]`	`True`	Membership
`for x in [1,2,3]: print(x)`	`1 2 3`	Iteration

- **A music player can keep a stack of visited views :**
  - `append` records the latest view; `pop` removes it when going back

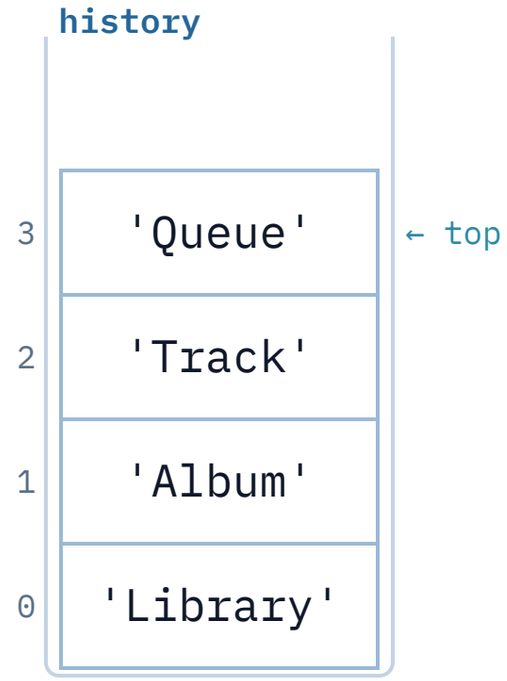
```
history = ["Library", "Album", "Track"]
history.append("Queue")
history.append("Artist")
print(history)  # ['Library', 'Album', 'Track', 'Queue', 'Artist']
print(history.pop())  # Artist
print(history)  # ['Library', 'Album', 'Track', 'Queue']
```

A List as a Stack

```
history = ["Library", "Album", "Track"]
history.append("Queue")
history.append("Artist")
print(history.pop())  # Artist
print(history)  # ['Library', 'Album', 'Track', 'Queue']
```

OUTPUT

```
Artist
['Library', 'Album', 'Track', 'Queue']
```



Index `0` is the bottom of the stack

List Functions and Methods

Function	Description
`len(list)`	Gives the total length of the list
`max(list)` / `min(list)`	Returns item from the list with max or min value
`list(seq)`	Converts a tuple into list
`list.append(obj)`	Appends object obj to list
`list.count(obj)`	Returns count of how many times obj occurs in list
`list.extend(seq)`	Appends the contents of seq to list
`list.index(obj)`	Returns the lowest index in list that obj appears
`list.insert(index, obj)`	Inserts object obj into list at offset index
`list.pop(index=-1)`	Removes and returns the item at an index; default is last
`list.remove(obj)`	Removes object obj from list
`list.reverse()`	Reverses objects of list in place
`list.sort(key=None, reverse=False)`	Sorts in place; options are keyword-only

demo05 · listmethods

SpiralTrain developer courses

Tuples

- **A tuple stores a fixed sequence of values :**
  - Each position can have a particular role, such as title and duration
  - Photo dimensions similarly pair width and height

```
track = ("Imagine", 183)
dimensions = (1920, 1080)
titles = "Imagine", "Yesterday"  # The comma creates a tuple.
empty = ()
single = ("Imagine",)  # A one-item tuple needs the comma.
```

- **Items cannot be replaced, inserted or removed from an existing tuple**
- **Commas create the tuple; parentheses usually make its boundaries clearer**
- **A tuple may contain mutable objects, whose own contents can still change**

SpiralTrain developer courses

Accessing Tuples

- Positions in this track record hold the title, artist and duration :

```
track = ("Imagine", "John Lennon", 183)
print(track[0])  # Imagine
print(track[1:])  # ('John Lennon', 183)
print(len(track))  # 3
# track[2] = 240  # TypeError: tuple items cannot be assigned.
```

- Slicing and concatenation can create a different record :

```
long_mix = track[:2] + (240,)
print(long_mix)  # ('Imagine', 'John Lennon', 240)
print(track)  # ('Imagine', 'John Lennon', 183)
```

- The original tuple remains unchanged

SpiralTrain developer courses

Delete Tuple Elements

- Individual items cannot be removed from an existing tuple
- A slice can create a shorter tuple :

```
titles = ("Imagine", "Yesterday", "Respect")
shorter = titles[:-1]
print(shorter)  # ('Imagine', 'Yesterday')
```

- `del` removes a name binding, not items from a tuple :

```
del titles
print(shorter)  # ('Imagine', 'Yesterday')
```

- Other names keep their objects available after a binding is removed

SpiralTrain developer courses

Usage of Tuples

- **Tuples have many uses :**
  - Photo dimensions, track metadata and records from a database
- **Any set of comma-separated objects default to tuples :**
  - Even without parentheses around them
- **Tuples can be dictionary keys and set members if every item is hashable**
- **Tuples can make your code safer :**
  - Allows you to "write-protect" data that does not need to be changed
- **A tuple can have lists as items :**
  - The items in the lists are mutable however
- **Tuple can be destructured and contents assigned to variables :**

```
selection = ("Imagine", "Toxic")
print(selection)  # ('Imagine', 'Toxic')
primary, backup = selection
print(primary)  # 'Imagine'
print(backup)  # 'Toxic'
```

demo08 · destructurewithtuples

SpiralTrain developer courses

Tuple Unpacking

```python
selection = ("Imagine", "Toxic")
primary, backup = selection
primary, backup = backup, primary
print(primary)  # 'Toxic'
print(backup)  # 'Imagine'
```

```
Toxic
Imagine
```

demo08 · destructurewithtuples

SpiralTrain developer courses

Unpacking in for Loops

- A `for` loop unpacks each item the same way, and the unpacking can nest :

```
rows = [(1,), (2,)]
for (number,) in rows:
    print(number)  # 1, then 2

records = [(1, ("Imagine", "John Lennon"))]
for number, (title, artist) in records:
    print(number, title, artist)  # 1 Imagine John Lennon
```

Tuple Functions

- **Only functions and methods that do not modify tuple available**

Function	Description
`len(tuple)`	Gives the total length of the tuple
`max(tuple)`	Returns item from the tuple with max value
`min(tuple)`	Returns item from the tuple with min value
`tuple(seq)`	Converts a list into tuple
`tup.index(i)`	Index of the first occurrence of i in tup
`tup.count(i)`	Total number of occurrences of i in tup

Sets

- **A set keeps each distinct value once :**
  - Repeated artist names or track requests become single members

```
artists = {"Queen", "Queen", "ABBA"}  # Duplicate names become one member.
print(sorted(artists))  # ['ABBA', 'Queen']
requests = ["Yesterday", "Yesterday", "Respect", "Respect"]
unique = set(requests)
unique.add("Imagine")
print(sorted(unique))  # ['Imagine', 'Respect', 'Yesterday']
print("Respect" in unique)  # True
```

- **Sets have no sequence order; `sorted` gives predictable printed output**
- **An empty set is `set()`; `{}` creates an empty dictionary**
- **`frozenset(requests)` creates a set whose members cannot be changed**

Set Operations

- Two fields of science show who worked in either field or in both :

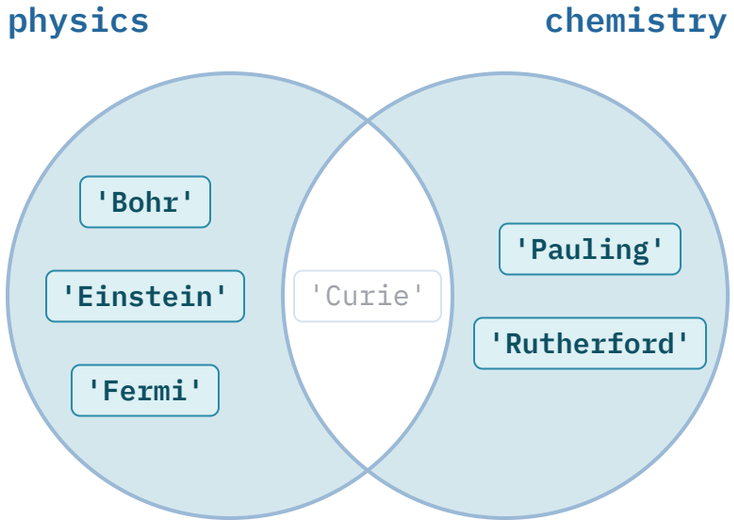
```
physics = {"Einstein", "Bohr", "Curie", "Fermi"}
chemistry = {"Curie", "Rutherford", "Pauling"}
print("Curie" in chemistry)  # True
print(sorted(physics | chemistry))  # union, in either set
print(sorted(physics & chemistry))  # intersection, in both
print(sorted(physics - chemistry))  # difference, in physics only
print(sorted(physics ^ chemistry))  # in one of the two, not both
```

```
True
['Bohr', 'Curie', 'Einstein', 'Fermi', 'Pauling', 'Rutherford']
['Curie']
['Bohr', 'Einstein', 'Fermi']
['Bohr', 'Einstein', 'Fermi', 'Pauling', 'Rutherford']
```

- A set has no order, so `sorted()` turns it into a list to print it predictably
- Each operator has a method form that accepts any iterable :
  - `union()`, `intersection()`, `difference()` and `symmetric_difference()`

Set Operations by Region

```
physics = {"Einstein", "Bohr", "Curie", "Fermi"}
chemistry = {"Curie", "Rutherford", "Pauling"}
print("Curie" in chemistry)  # True
print(sorted(physics | chemistry))  # union, in either set
print(sorted(physics & chemistry))  # intersection, in both
print(sorted(physics - chemistry))  # difference, in physics only
print(sorted(physics ^ chemistry))  # in one of the two, not both
```



OUTPUT

```
True
['Bohr', 'Curie', 'Einstein', 'Fermi', 'Pauling', 'Rutherford']
['Curie']
['Bohr', 'Einstein', 'Fermi']
['Bohr', 'Einstein', 'Fermi', 'Pauling', 'Rutherford']
```

`^` keeps the names that are in exactly one of the two sets

demo09 · setsandfrozensets

SpiralTrain developer courses

Dictionaries

- **A dictionary can map a track title to its duration in seconds :**
  - Entries consist of keys and values
  - Keys must be hashable; strings and numbers are common choices
  - Values can be any arbitrary Python object
- **Dictionaries are hash tables :**
  - Looking up a key is fast, however large the dictionary
- **Dictionaries are enclosed by curly braces `{ }` :**

```
durations = {"Imagine": 183, "Yesterday": 125, "Respect": 149}
print(durations)  # {'Imagine': 183, 'Yesterday': 125, 'Respect': 149}
```

- **Values can be assigned and accessed using square brackets `[]` :**

```
durations["Toxic"] = 199
durations["Thriller"] = 357
print(durations)
# {'Imagine': 183, 'Yesterday': 125, 'Respect': 149, 'Toxic': 199, 'Thriller': 357}
```

- **Dictionaries remember insertion order :**
  - Keys come out in the order in which they were first added

SpiralTrain developer courses

Accessing Values in Dictionary

- **Keys are unique within a dictionary while values may be the same :**
  - Values of a dictionary can be of any type
  - Keys must be hashable; a tuple qualifies only if all its items are hashable
- **Use square brackets along with the key to obtain its value :**

```
print(durations["Yesterday"])  # 125
print(durations["Thriller"])  # 357
```

- **Dictionary can be updated by modifying an existing entry :**

```
durations["Respect"] = 147
print(durations["Respect"])  # 147
```

- **Dictionary entry can be deleted :**

```
del durations["Imagine"]
print(durations["Imagine"])  # KeyError
```

- **Entire contents of dictionary is cleared with `clear()` :**
  - Entire dictionary can be deleted in single operation with del

Updating a Dictionary

```
durations = {"Imagine": 183, "Yesterday": 125, "Respect": 149}
durations["Toxic"] = 199
durations["Thriller"] = 357
print(durations["Yesterday"])  # 125
durations["Respect"] = 147
del durations["Imagine"]
```

OUTPUT

```
125
```

durations

KEY	VALUE
'Yesterday'	125
'Respect'	147
'Toxic'	199
'Thriller'	357

removed

~~'Imagine'~~	183

`del` removes the key and its value

SpiralTrain developer courses

Properties of Dictionary Keys

- **Keys must be hashable :**
  - Their hash stays stable, and equal keys must have equal hashes
  - Strings and numbers qualify; lists and dictionaries do not
  - A tuple is hashable only when every item is hashable
- **Values can be any object; assigning an equal key replaces its value**

```
track_lengths = {("Imagine", "John Lennon"): 183}
print(track_lengths[("Imagine", "John Lennon")])  # 183
```

- **An immutable tuple can contain an unhashable list :**

```
hash(("Imagine", []))  # TypeError: unhashable type: 'list'
```

SpiralTrain developer courses

Annotating Collections

- Each built-in collection is annotated with the type of what it holds :

```
scores: list[int] = [10, 20, 30]
names: set[str] = {"Alice", "Bob"}
track_lengths: dict[str, int] = {"Imagine": 183, "Yesterday": 125}
```

- A dictionary takes two types, the key first and then the value
- A tuple is annotated position by position, because its length is fixed :

```
point: tuple[int, int] = (3, 4)
track: tuple[str, int] = ("Imagine", 183)
```

- An ellipsis means any number of one type :

```
readings: tuple[float, ...] = (1.5, 2.5, 3.5)
```

- Annotate where the collection is homogeneous, which most are :
  - Mixed element types are allowed, but a playlist of titles has the clearer annotation `list[str]`

Dictionary Functions and Methods

Function	Description
`len(dict)`	Gives the total length, or nr. of items, in the dictionary
`str(dict)`	Produces a printable string representation of a dictionary
`fromkeys()`	Create new dictionary with keys from seq, values set to value
`dict.clear()`	Removes all elements of dictionary dict
`dict.copy()`	Returns a shallow copy of dictionary dict
`dict.get(key)`	For key key, returns value or default if key not in dictionary
`dict.popitem()`	Removes and returns the (key, value) pair added last
`dict.items()`	Returns a view of dict'backup (key, value) tuple pairs
`dict.keys()`	Returns a view of dictionary dict'backup keys
`dict.update(dict2)`	Adds dictionary dict2'backup key-values pairs to dict
`dict.values()`	Returns a view of dictionary dict'backup values

SpiralTrain developer courses

Summary : Data Structures

- **Sequences are most basic data structure :**
  - Each element of a sequence is assigned a number, its position or index
  - First index is zero, the second index is one, and so forth
- **A playlist stores tracks in a chosen order :**
  - Lists are mutable sequences
  - Contain items separated by commas and enclosed within square brackets `[]`
  - Items can be of different data types, and lists can hold other lists
- **Tuples are immutable sequences :**
  - Enclosed in parenthesis ( ) instead of brackets
  - Consist of a number of values separated by commas
- **Sets are unordered collections with no duplicates :**
  - Fast membership tests, and union, intersection and difference
  - Set is mutable and `frozenset` is immutable
- **Dictionaries map keys to values, in insertion order :**
  - Pairs of keys and values, separated by a comma and a colon
  - Enclosed by curly braces `{ }`, values accessed with square brackets `[]`

Control Flow

- Python Intro
- Variables and Data Types
- Data Structures
- Control Flow
- Functions
- Modules
- Comprehensions
- Exception Handling
- Python IO
- Python Database Access
- Python Classes
- APPENDIX Type Hinting
- APPENDIX Python Libraries

- Control Flow Constructs
- if Statement
- if Example
- else Statement
- elif Statement
- Nested if...elif...else
- while Loop
- Infinite while Loop
- for Loop
- Looping over a Playlist
- Iterating by Sequence Index
- Iterating with enumerate
- break and continue Statements
- Loop with else Combination
- pass Statement

- match Statement
- Summary : Control Flow

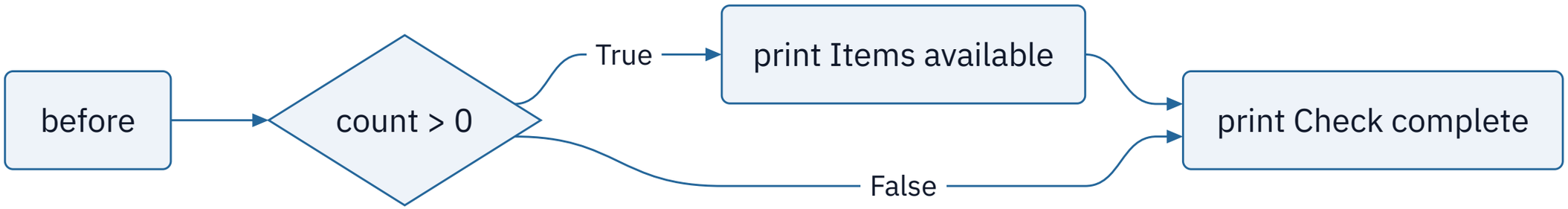
Control Flow Constructs

- **Statements run one after the other unless a control flow construct intervenes :**
  - Branching constructs choose which block runs
  - Iteration constructs repeat a block
- **Branching statements :**
  - `if`, `if...else` and `elif`
  - `match`, which compares one value against a series of patterns
- **Iteration statements :**
  - `while`, which repeats as long as a condition holds
  - `for`, which takes each item of a sequence in turn
- **Statements that steer a loop or fill a gap :**
  - `break`, `continue`, a loop's `else`, and `pass`

if Statement

- `if` runs an indented block when its condition is true.
- Execution continues after the block whether the condition is true or false.

```
count = 3
if count > 0:
    print("Items available")
print("Check complete")
```



- Four spaces of indentation group the statements in the block.
- `0`, `""`, empty collections and `None` also count as false in a condition.

if Example

- A camera checks whether storage has room for another photo :

```
photos_left = 3
if photos_left > 0:
    print("Photo space available")
    print(f"Room for {photos_left} photos")
print("Camera check complete")
```

```
Photo space available
Room for 3 photos
Camera check complete
```

- With `photos_left = 0`, the indented block is skipped
- The final message appears in both cases

else Statement

- `else` runs when the `if` condition is false :
  - Exactly one of the two blocks runs
  - `else` has no condition of its own

```
battery = 15
if battery >= 20:
    print("Camera ready")
else:
    print("Charge the battery")
```

```
Charge the battery
```

- With `battery = 80`, the first block runs instead
- An optional `else` comes last

demo02 · else

elif Statement

- `elif` **tests another condition when earlier conditions were false :**
  - Only the first matching block runs
  - An optional `else` handles anything left over

```
filename = "cover.png"
if filename.endswith(".jpg"):
    print("JPEG photo")
elif filename.endswith(".png"):
    print("PNG image")
else:
    print("Unsupported image format")
```

```
PNG image
```

Nested if...elif...else

- An inner condition is tested only when its outer block runs :
  - The storage check runs only when the battery has enough charge

```
battery = 80
photos_left = 3
if battery >= 20:
    if photos_left > 0:
        print("Camera ready")
    else:
        print("Storage full")
else:
    print("Charge the battery")
```

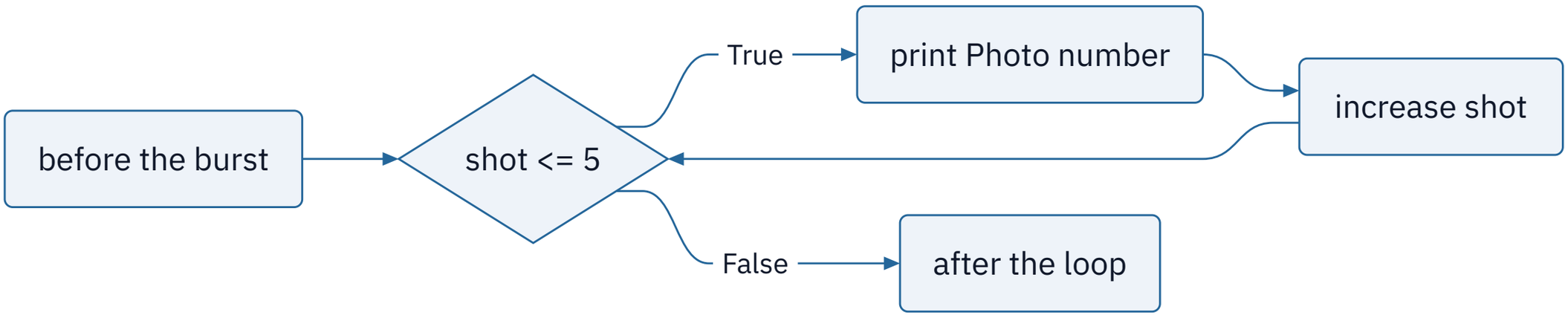
- Each nesting level adds four spaces
- A single test can combine these conditions when separate messages are unnecessary :
  - `battery >= 20 and photos_left > 0`

while Loop

- `while` **repeats a block while its condition is true :**
  - The condition is tested before every pass, including the first

```
shot = 1
while shot <= 5:
    print(f"Photo {shot}")
    shot += 1
```

- **This photo burst prints** `Photo 1` **through** `Photo 5`
- `shot += 1` **changes the value used by the next condition test**
- **When** `shot` **reaches** `6`**, execution continues after the loop**

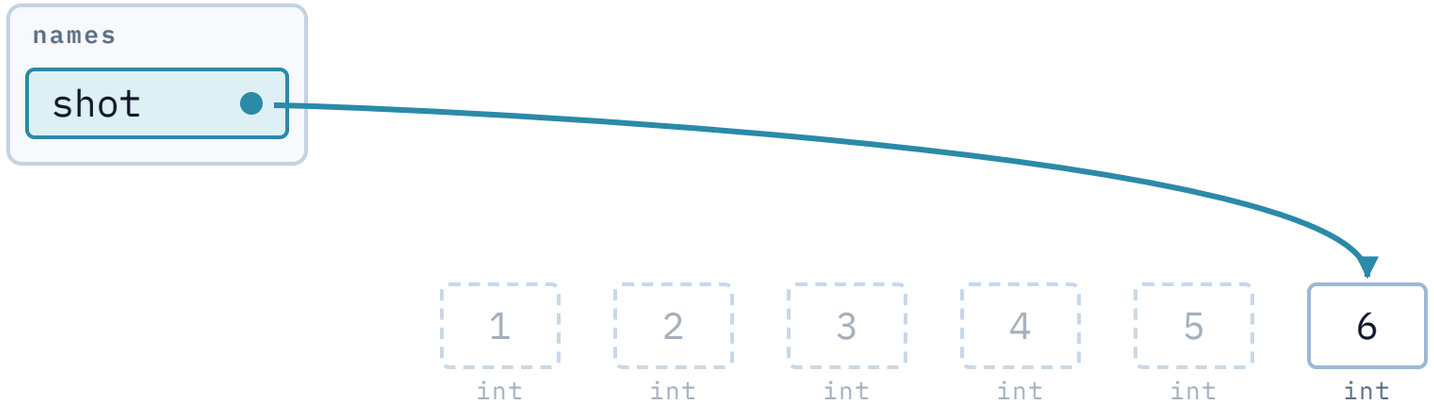


The Condition Before Each Pass

```
shot = 1
while shot <= 5:
    print(f"Photo {shot}")
    shot += 1
```

OUTPUT

```
Photo 1
Photo 2
Photo 3
Photo 4
Photo 5
```



`6 <= 5` is false, so the loop ends without running the block

Infinite while Loop

- A player can keep accepting requests until interrupted :

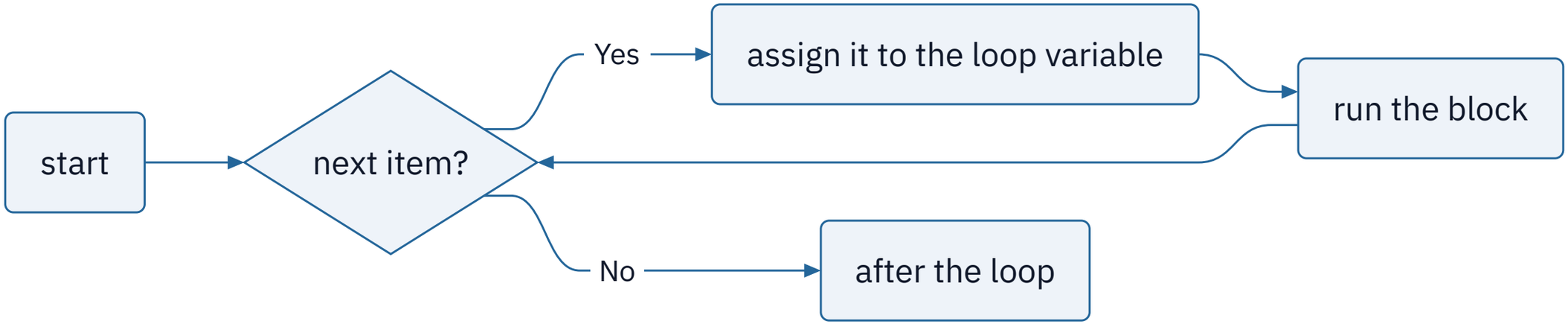
```
while True:
    title = input("Queue a track: ")
    print(f"Requested: {title}")
```

- `True` never becomes false, so this loop does not finish by itself
- Press Ctrl+C to interrupt it; Python reports `KeyboardInterrupt`
- A condition with `break` can provide an ordinary exit from a continuous loop

demo06 · infinitewhile

for Loop

- `for` runs a block once for each item in a list, string or other iterable.
- The loop variable receives the next item before the block runs.



- An empty sequence skips the block entirely.
- The loop stops automatically when there are no items left.

SpiralTrain developer courses

Looping over a Playlist

- `title` takes each item from `playlist` in turn.
- The unindented `print` runs once, after the loop.

```
playlist = ["Imagine", "Yesterday", "Respect"]
for title in playlist:
    print(f"Now playing: {title}")
print("Playlist finished")
```

```
Now playing: Imagine
Now playing: Yesterday
Now playing: Respect
Playlist finished
```

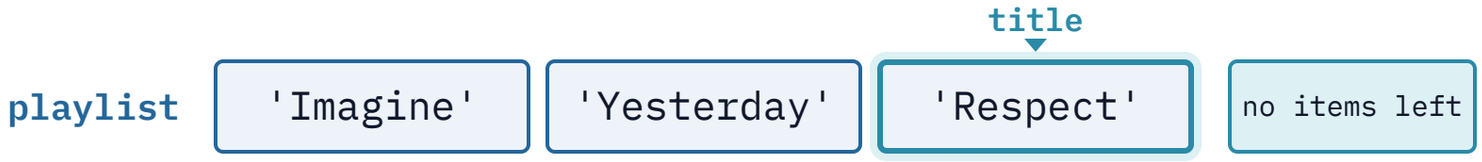
- Any iterable works the same way; a string gives one character per pass :

```
for letter in "ABBA":
    print(letter, end=" ")
```

```
A B B A
```

One Item per Pass

```
playlist = ["Imagine", "Yesterday", "Respect"]
for title in playlist:
    print(f"Now playing: {title}")
print("Playlist finished")
```



OUTPUT

```
Now playing: Imagine
Now playing: Yesterday
Now playing: Respect
Playlist finished
```

No items are left, so the loop ends and the unindented `print` runs

Iterating by Sequence Index

- **A sequence can be walked by index, using `len()` and `range()` :**

```
playlist = ["Imagine", "Yesterday", "Respect"]
for index in range(len(playlist)):
    print("Now playing :", playlist[index])
```

- **This works, and is what a loop over positions looks like in many languages :**
  - `len()` gives the number of elements, `range()` the sequence of positions
  - It is worth recognising, because you will meet it in other people's code
- **In Python it is rarely what you want :**
  - The index is only used to look the element back up again
  - A `range` that drifts out of step with the sequence is a common bug
- **When the element is all you need, iterate the sequence itself :**

```
for title in playlist:
    print("Now playing :", title)
```

Iterating with enumerate

- When the index is genuinely needed, `enumerate` yields both :

```
for index, title in enumerate(playlist):
    print(index, title)
```

```
0 Imagine
1 Yesterday
2 Respect
```

- The two can never fall out of step, because they come from one call
- Counting starts at zero unless a `start` is given :

```
for number, title in enumerate(playlist, start=1):
    print(number, title)
```

- The pattern generalises :
  - `enumerate(seq)` for position and element
  - `zip(a, b)` for two sequences walked in step, covered in Comprehensions
  - `dict.items()` for a dictionary’s keys and values together

break and continue Statements

- `continue` skips an item and starts the next iteration :

```
for filename in ["cover.jpg", "notes.txt", "stage.jpg"]:
    if not filename.endswith(".jpg"):
        continue
    print(f"Photo: {filename}")
```

- `break` ends the loop immediately when the requested track is found :

```
for track in ["Thriller", "Imagine", "Yesterday", "Respect", "Hey Jude"]:
    if track == "Respect":
        break
    print(f"Checked {track}")
print("Search complete")
```

- The final message runs after the search; `Hey Jude` is never checked

Leaving a Loop with break

```
for track in ["Thriller", "Imagine", "Yesterday", "Respect", "Hey Jude"]:
    if track == "Respect":
        break
    print(f"Checked {track}")
print("Search complete")
```

OUTPUT

```
Checked Thriller
Checked Imagine
Checked Yesterday
Search complete
```



Execution continues after the loop; `track` still refers to `Respect`

Loop with else Combination

- **A search uses `else` for the case where no item matched :**
  - The `else` block runs when the loop finishes without `break`

```
requested = "Wonderwall"
playlist = ["Imagine", "Yesterday", "Respect"]
for track in playlist:
    if track == requested:
        print(f"Found {requested}")
        break
else:
    print(f"{requested} is not in the playlist")
```

```
Wonderwall is not in the playlist
```

- **With `requested = "Yesterday"`, `break` skips the `else` block**
- **A `while` loop can also have `else`; it runs when the condition becomes false**

pass Statement

- `pass` **fills a required block while an action is still being written :**
  - It performs no operation
  - Execution continues with the next statement

```
command = "skip"
if command == "skip":
    pass  # The next-track action is not implemented yet.
print(f"Command received: {command}")
```

```
Command received: skip
```

- **Unlike** `continue`**,** `pass` **does not skip the rest of a loop iteration**

match Statement

- `match` compares a value with the cases in order :
  - Only the first matching case runs

```
command = "play"
match command:
    case "play":
        print("Playing the current track")
    case "skip":
        print("Moving to the next track")
    case "quit":
        print("Player stopped")
    case _:
        print("Unknown player command")
```

- Quoted strings match those exact values
- `case _` handles remaining values and belongs last
- Execution continues after the block; cases need no `break`
- Python 3.10 or later is required

SpiralTrain developer courses

Summary : Control Flow

- **Branching statements choose which code runs :**
  - `if`, `if...else` and `elif` test conditions in turn
  - `match` compares data against a series of patterns
- **Iteration statements repeat a block of code :**
  - `while` repeats as long as a condition holds
  - `for` iterates over each item of a sequence
- **Statements that steer a loop :**
  - `break` exits the loop early
  - `continue` skips to the next iteration
  - An `else` clause runs when the loop finishes without a `break`
- **The `pass` statement is a no-op placeholder where a statement is required**

Functions

Python Intro

Variables and Data Types

Data Structures

Control Flow

Functions

Modules

Comprehensions

Exception Handling

Python IO

Python Database Access

Python Classes

APPENDIX Type Hinting

APPENDIX Python Libraries

Functions

Function Syntax

Calling Functions

return Statement

Functions Without a Result

Annotating a Function

Documenting a Function

Function Arguments

Keyword Arguments

Default Arguments

Passing Object References

Overwriting References

Mutating Versus Rebinding

Variable Length Arguments

Lambda Functions

Functions in Dictionaries

Scope of Variables

Example Variable Scope

Summary : Functions

SpiralTrain developer courses

Functions

- **A function gives a name to a task that a program repeats :**
  - A music player stores track lengths in seconds
  - Its display needs a duration such as `3:42`
- **Python provides functions such as `print()`, `input()` and `len()` :**
  - Programs define their own functions with `def`
- **A function can take a value and return a result :**
  - The caller supplies the number of seconds
  - The function returns the display text

SpiralTrain developer courses

Function Syntax

- `format_duration` **turns a track length into display text :**
  - `// 60` gives complete minutes; `% 60` gives remaining seconds
  - `:02d` displays the seconds with two digits, including a leading zero

```
def format_duration(seconds):
    minutes = seconds // 60
    remainder = seconds % 60
    return f"{minutes}:{remainder:02d}"
```

- **A definition starts with** `def` **, the name and parentheses :**
  - `seconds` is a parameter that receives a track length
  - The colon starts the indented body
- `return` **passes the display text back to the caller**

Calling Functions

- **Define a function before calling it :**
  - Parentheses contain the argument supplied for `seconds`
  - Each call runs the same body with a different track length

```
print(format_duration(222))
print(format_duration(65))
```

```
3:42
1:05
```

- **Defining the function does not run its body**
- **Omitting a required argument raises `TypeError`**

SpiralTrain developer courses

return Statement

- **A returned value can be stored and used by the caller :**
  - `label` receives the duration text
  - The caller includes it in a track description

```
def format_duration(seconds):
    minutes = seconds // 60
    remainder = seconds % 60
    return f"{minutes}:{remainder:02d}"

label = format_duration(125)
message = f"Yesterday [{label}]"
print(message)
```

```
Yesterday [2:05]
```

- `return` **ends the current call and passes its value back**

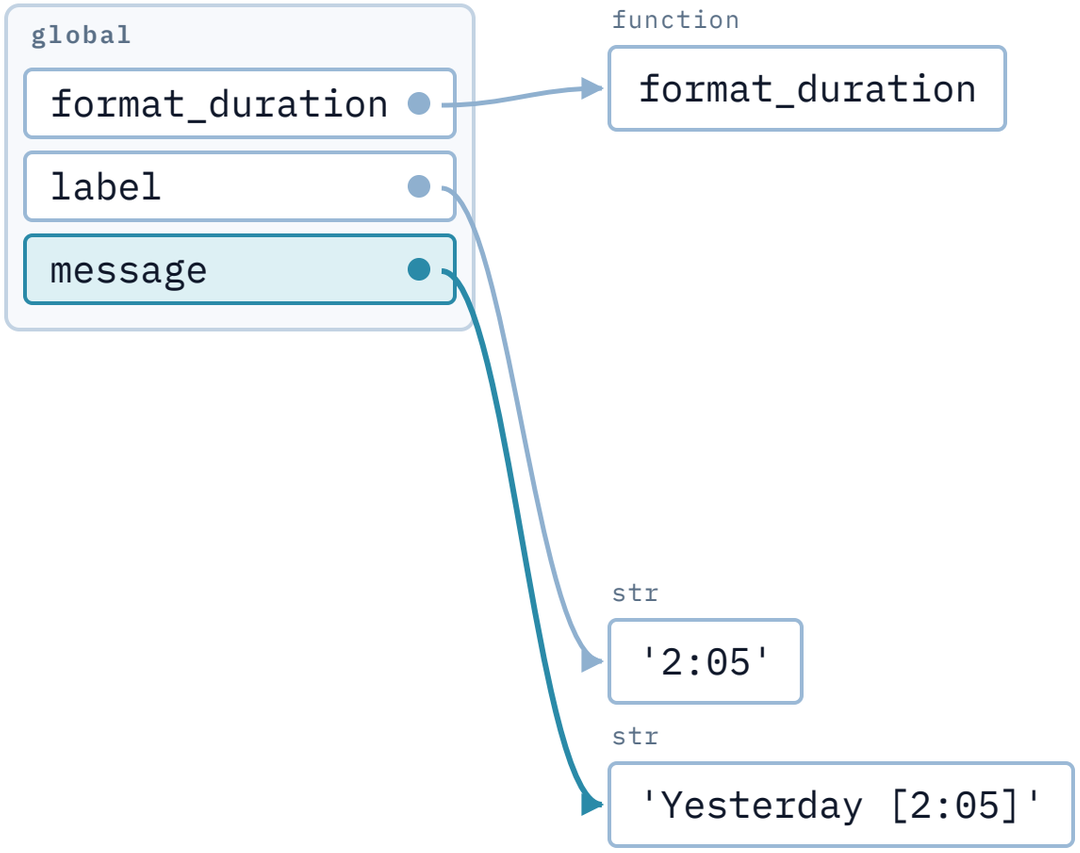
Using the Returned Duration

```
def format_duration(seconds):
    minutes = seconds // 60
    remainder = seconds % 60
    return f"{minutes}:{remainder:02d}"

label = format_duration(125)
message = f"Yesterday [{label}]"
print(message)
```

OUTPUT

```
Yesterday [2:05]
```



The message is printed; the function binding and caller variables remain

SpiralTrain developer courses

Functions Without a Result

- **Some functions perform a task without returning a useful value :**
  - `print_playlist` prints a heading and numbered tracks
  - The caller does not need to store a result

```
def print_playlist(name, tracks):
    print(f"Playlist: {name}")
    for number, title in enumerate(tracks, start=1):
        print(f"{number}. {title}")


print_playlist("Night drive", ["Imagine", "Yesterday"])
```

```
Playlist: Night drive
1. Imagine
2. Yesterday
```

- **Reaching the end of a function returns `None`**
- **A bare `return` also returns `None`**

demo03 · playlist

Annotating a Function

- **Annotations describe the values a function expects and returns :**
  - `seconds: int` describes the argument; `-> str` describes the result

```
def format_duration(seconds: int) -> str:
    minutes = seconds // 60
    remainder = seconds % 60
    return f"{minutes}:{remainder:02d}"
```

- **`-> None` describes a function that does not return a useful value :**

```
def print_playlist(name: str, tracks: list[str]) -> None:
    print(f"Playlist: {name}")
    for number, title in enumerate(tracks, start=1):
        print(f"{number}. {title}")
```

- **Annotations help editors and checkers; Python does not enforce them**

SpiralTrain developer courses

Documenting a Function

- **A docstring describes the task and rules callers need to know :**
  - It is the first string inside the function body

```
def format_duration(seconds: int) -> str:
    """Format a non-negative track length as minutes and two-digit seconds."""
    minutes = seconds // 60
    remainder = seconds % 60
    return f"{minutes}:{remainder:02d}"
```

- `help(format_duration)` **displays the docstring**
- **Triple quotes allow a description to span lines**
- **Documentation tools can include docstrings in generated reference pages**

SpiralTrain developer courses

Function Arguments

- **Positional arguments follow the parameter order :**
  - `seconds` receives the first value; `separator` receives the second

```
def format_duration(seconds: int, separator: str) -> str:
    minutes = seconds // 60
    remainder = seconds % 60
    return f"{minutes}{separator}{remainder:02d}"


print(format_duration(222, ":"))
```

- **Keyword arguments specify parameter names**
- **Default values allow arguments to be omitted**
- **`*args` and `**kwargs` collect extra positional and keyword arguments**
- **Defining a name again replaces its earlier definition**

SpiralTrain developer courses

Keyword Arguments

- **Keyword arguments identify parameters by name :**
  - Their order can differ from the definition

```
def format_duration(seconds: int, separator: str) -> str:
    minutes = seconds // 60
    remainder = seconds % 60
    return f"{minutes}{separator}{remainder:02d}"


print(format_duration(separator=".", seconds=222))
```

```
3.42
```

- **Positional arguments must come before keyword arguments**
- **Names make the purpose of each argument visible**

Default Arguments

- **A default supplies a value when an argument is omitted :**
  - Track durations usually separate minutes and seconds with `:`
  - A caller can request another separator

```
def format_duration(seconds: int, separator: str = ":") -> str:
    minutes = seconds // 60
    remainder = seconds % 60
    return f"{minutes}{separator}{remainder:02d}"


print(format_duration(222))
print(format_duration(222, separator="m "))
```

```
3:42
3m 42
```

- **Parameters without defaults cannot follow parameters with defaults**

Passing Object References

- **A parameter refers to the same object as the caller's argument :**
  - `add_encore` adds a final track to the shared queue

```
def add_encore(tracks: list[str]) -> None:
    tracks.append("Let It Be")
    print(f"Inside: {tracks}")

queue = ["Imagine", "Yesterday"]
add_encore(queue)
print(f"After call: {queue}")
```

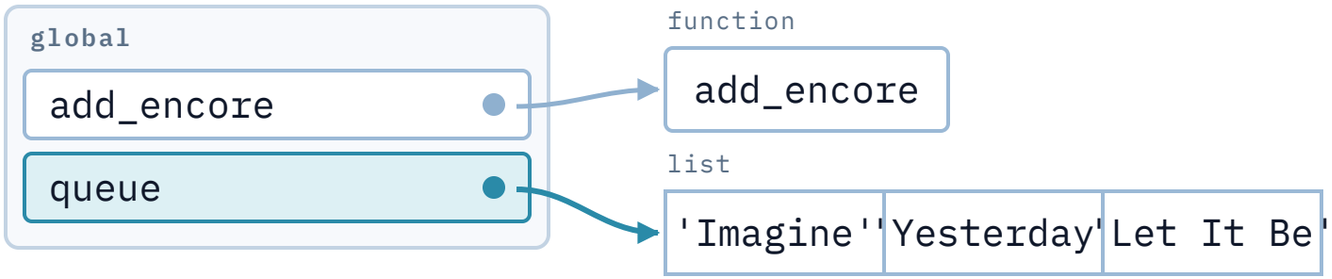
```
Inside: ['Imagine', 'Yesterday', 'Let It Be']
After call: ['Imagine', 'Yesterday', 'Let It Be']
```

- `append` **changes the existing list, so the caller sees the encore**

Adding an Let It Be

```python
def add_encore(tracks: list[str]) -> None:
    tracks.append("Let It Be")
    print(f"Inside: {tracks}")

queue = ["Imagine", "Yesterday"]
add_encore(queue)
print(f"After call: {queue}")
```



OUTPUT

```
Inside: ['Imagine', 'Yesterday', 'Let It Be']
After call: ['Imagine', 'Yesterday', 'Let It Be']
```

The caller sees the added track in `queue`

Overwriting References

- **A function can create an encore mix while keeping the original queue :**
  - `tracks + ["Let It Be"]` creates a new list
  - Assigning to `tracks` changes only the local name

```
def with_encore(tracks: list[str]) -> list[str]:
    tracks = tracks + ["Let It Be"]
    return tracks

queue = ["Imagine", "Yesterday"]
extended = with_encore(queue)
print(f"Original: {queue}")
print(f"With encore: {extended}")
```

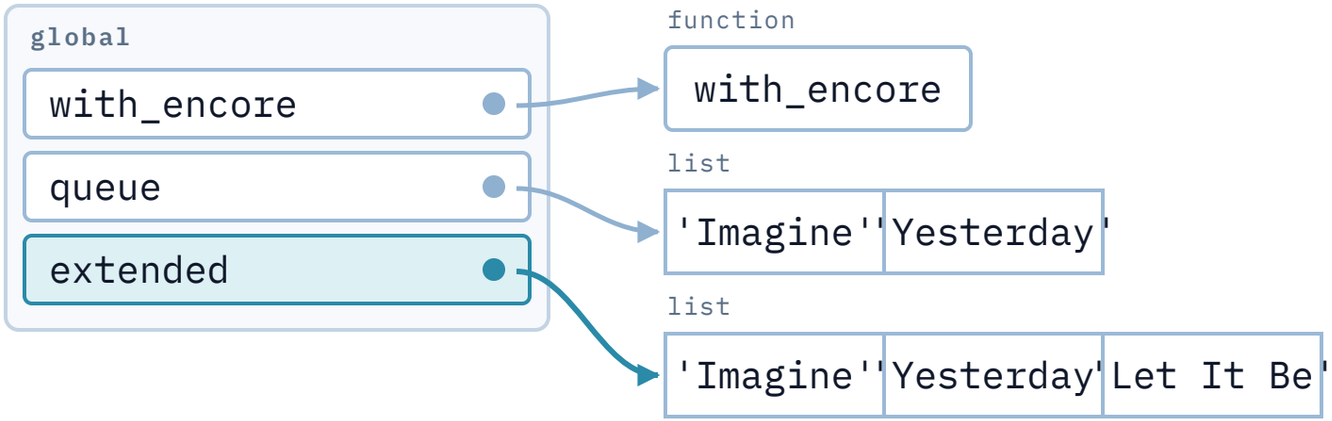
```
Original: ['Imagine', 'Yesterday']
With encore: ['Imagine', 'Yesterday', 'Let It Be']
```

- **The caller stores the returned list in `extended`; `queue` keeps its original list**

Keeping the Original Playlist

```
def with_encore(tracks: list[str]) -> list[str]:
    tracks = tracks + ["Let It Be"]
    return tracks

queue = ["Imagine", "Yesterday"]
extended = with_encore(queue)
print(f"Original: {queue}")
print(f"With encore: {extended}")
```



OUTPUT

```
Original: ['Imagine', 'Yesterday']
With encore: ['Imagine', 'Yesterday',
'Let It Be']
```

The encore mix uses the new list including `Let It Be`

SpiralTrain developer courses

Mutating Versus Rebinding

- Both functions add an encore, with different effects on the original queue :

Operation inside the function	Effect
`tracks.append("Let It Be")`	Adds to the list the caller already holds
`tracks = tracks + ["Let It Be"]`	Binds the local name to a new list
`return tracks`	Passes that new list back to the caller

- Mutate a shared queue when every caller should see the change
- Return a new mix when the original should stay available
- Integers and strings cannot be changed in place; a different value needs a new binding

Variable Length Arguments

- `*tracks` collects extra positional arguments into a tuple
- `**details` collects extra keyword arguments into a dictionary

```
def show_playlist(name: str, *tracks: str, **details: str) -> None:
    print(f"Playlist: {name}")
    for track in tracks:
        print(f"- {track}")
    for key, value in details.items():
        print(f"{key}: {value}")


show_playlist("Night drive", "Imagine", "Yesterday", mood="calm", owner="Sam")
```

- At a call, `*values` unpacks a sequence and `**options` unpacks a dictionary
- `args` and `kwargs` are common names; task-specific names describe their contents

demo11 · variableargumentlist

demo12 · callingwithstar

Lambda Functions

- **A lambda is a small function passed directly to another function :**
  - `sorted` calls the key function for each track
  - `track[1]` selects its duration in seconds

```
tracks = [("Imagine", 183), ("Yesterday", 125), ("Respect", 149)]
by_length = sorted(tracks, key=lambda track: track[1])
print(by_length)
```

```
[('Yesterday', 125), ('Respect', 149), ('Imagine', 183)]
```

- **The value after the colon is returned automatically**
- **Use a named function when the calculation needs several lines**

Functions in Dictionaries

- **Functions are values that can be stored and called from a dictionary :**
  - The key selects which title formatter runs

```
def clean_title(title: str) -> str:
    return title.strip()


def shout_title(title: str) -> str:
    return title.upper()


formatters = {"clean": clean_title, "shout": shout_title}
print(formatters["clean"]("  Imagine  "))
for name, format_title in formatters.items():
    print(name, format_title("Yesterday"))
```

```
Imagine
clean Yesterday
shout YESTERDAY
```

- **A dictionary mapping keys to functions is a dispatch table**

SpiralTrain developer courses

Scope of Variables

- **Scope determines where an identifier can be accessed :**
  - Variables created inside a function are local to that function
  - Variables created outside any function belong to the module's global scope
- **Python looks up names in four scopes, in LEGB order :**



- **The first scope that contains the name wins; the rest are not searched**
- **Local variables :**
  - Exist only while the function is executing
  - Cannot be read or modified from outside the function
- **Global variables :**
  - Can be read from anywhere within the module
  - Rebinding a global name inside a function requires `global`
  - Mutating the object it names, such as calling `items.append(x)`, does not

Example Variable Scope

- **A local assignment affects only the current call :**
  - `preview_volume` shows a local volume without changing the configured value
- **`global` makes an assignment update the module's name :**
  - `set_limit` changes the shared queue limit

```
volume = 50
queue_limit = 20

def preview_volume() -> None:
    volume = 10
    print(f"Preview volume: {volume}")


def set_limit() -> None:
    global queue_limit
    queue_limit = 30

preview_volume()
print(f"Current volume: {volume}")
set_limit()
print(f"Queue limit: {queue_limit}")
```

demo15 · variablescope

Local and Shared Player Settings

```
volume = 50
queue_limit = 20

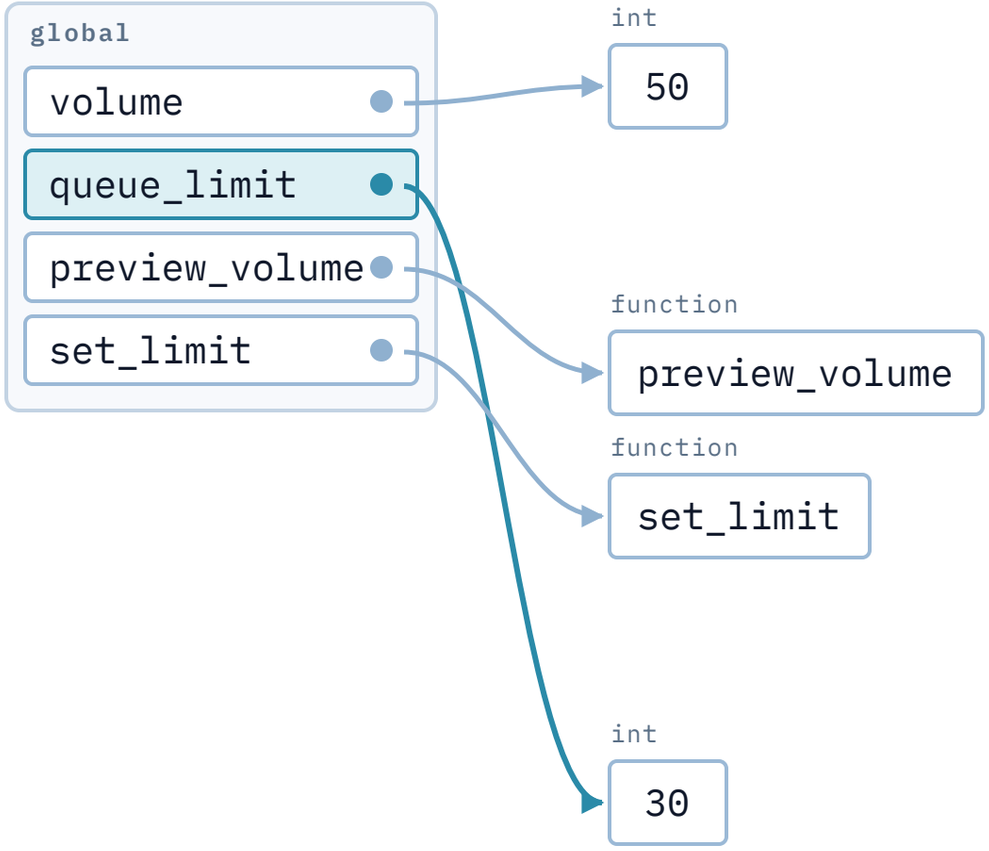
def preview_volume() -> None:
    volume = 10
    print(f"Preview volume: {volume}")

def set_limit() -> None:
    global queue_limit
    queue_limit = 30

preview_volume()
print(f"Current volume: {volume}")
set_limit()
print(f"Queue limit: {queue_limit}")
```

OUTPUT

```
Preview volume: 10
Current volume: 50
Queue limit: 30
```



The queue limit is now `30`

SpiralTrain developer courses

Summary : Functions

- **Functions are reusable blocks of code defined with `def` :**
  - Parameters and return types are documented with type annotations
- **Python passes arguments by object reference :**
  - Mutating an argument affects the caller; rebinding a parameter name does not
- **Flexible argument handling :**
  - Positional, keyword, default values, `*args` and `**kwargs`
- **Small anonymous functions use `lambda` :**
  - Ideal as short key functions passed to `sorted()`
- **Functions are values :**
  - A dictionary of functions is a dispatch table, `table[key](argument)`
- **Variables follow LEGB scope lookup :**
  - Local, Enclosing, Global and Built-in scopes

Modules

Python Intro
Variables and Data Types
Data Structures
Control Flow
Functions
Modules
Comprehensions
Exception Handling
Python IO
Python Database Access
Python Classes
APPENDIX Type Hinting
APPENDIX Python Libraries

Modules
import Statement
from...import Statement
Locating Modules
Using Modules
dir Function
Packages in Python
Explicit Import of Package Modules
Importing Names from a Package
Running a Module Directly
Running Packages with python -m
Installing Packages
Virtual Environments
Summary : Modules

Modules

- A module is a Python file containing related functions and variables
- Other files import the module to reuse those definitions
- `timing.py` holds the duration formatter used by a music player :

```
SECONDS_PER_MINUTE: int = 60


def format_duration(seconds: int) -> str:
    minutes = seconds // SECONDS_PER_MINUTE
    remainder = seconds % SECONDS_PER_MINUTE
    return f"{minutes}:{remainder:02d}"
```

- The module name is the filename without `.py`
- Keep `timing.py` beside the script that imports it

import Statement

- `import timing` loads the module and binds the name `timing`.
- The module prefix identifies where a function or constant comes from.
- `as` gives a module an alternative name in the calling file.

```
import timing

print(timing.format_duration(222))  # 3:42
print(timing.format_duration(65))  # 1:05

import timing as clock

print(clock.SECONDS_PER_MINUTE)  # 60
```

Importing a Module

```
import timing

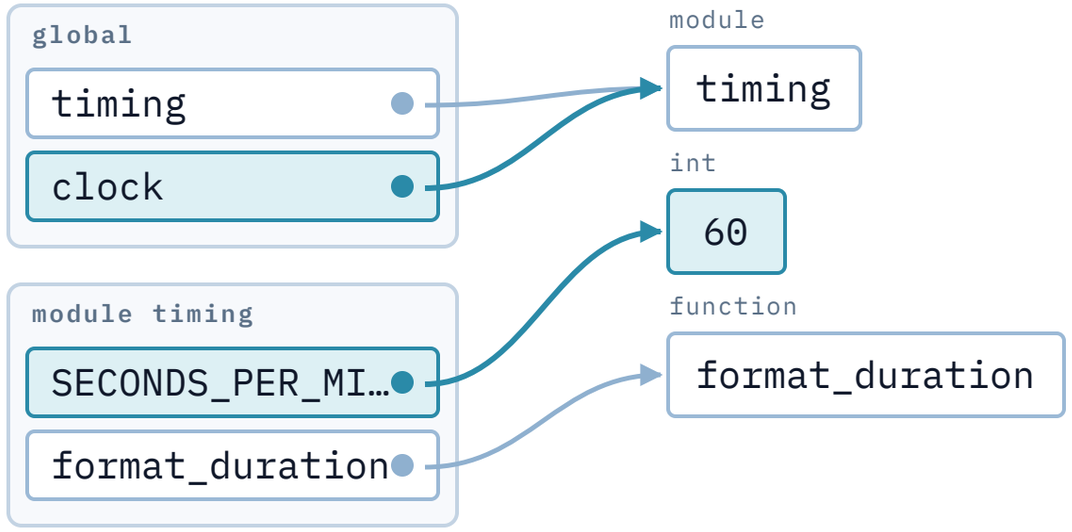
print(timing.format_duration(222))  # 3:42
print(timing.format_duration(65))  # 1:05

import timing as clock

print(clock.SECONDS_PER_MINUTE)  # 60
```

OUTPUT

```
3:42
1:05
60
```



`clock.SECONDS_PER_MINUTE` reaches the module's `SECONDS_PER_MINUTE`

demo01 · importmodule

from...import Statement

- `from ... import` binds selected names directly in the calling file.
- Call the function without the module prefix.
- Both import forms load the same module; they differ in the names they bind.
- Explicit names keep dependencies visible; avoid `from timing import *`.

```
from timing import SECONDS_PER_MINUTE, format_duration

print(format_duration(222))  # 3:42
print(SECONDS_PER_MINUTE)  # 60
```

demo02 · usingfromimport

SpiralTrain developer courses

Locating Modules

- Python searches the directories in `sys.path` for local modules.
- A script’s directory is searched before installed packages.
- With `python -m`, the current directory is searched first.
- Keep `timing.py` beside the script that imports it.
- A file named `json.py` can hide the standard library’s `json` module.
- `ModuleNotFoundError` means the requested module could not be found.

```
import sys

print(sys.path[0])
print(sys.executable)
```

demo03 · pythonpath

Using Modules

- **A playlist display reuses the formatter without copying its calculation :**
  - The calling script supplies track titles and lengths
  - The module formats each length consistently

```
from timing import format_duration

tracks = {"Imagine": 183, "Yesterday": 125}
for title, seconds in tracks.items():
    label = format_duration(seconds)
    print(f"{title}: {label}")
```

```
Imagine: 3:03
Yesterday: 2:05
```

- **Updating the formatter in one module updates every caller**

dir Function

- `dir(module)` lists the names available on a module.
- Explore a module in the REPL before calling an unfamiliar function.
- Names starting and ending with double underscores include module metadata.

```
import timing

print("format_duration" in dir(timing))  # True
print("SECONDS_PER_MINUTE" in dir(timing))  # True
```

demo05 · usingdir

Packages in Python

- A package groups related modules in one directory.
- A regular package contains an `__init__.py` file; it may be empty.

```
player/
├── __init__.py
├── timing.py     the format_duration function
└── __main__.py     the command-line entry point
```

- Move the existing `timing.py` into `player/` to group reporting code.
- `__init__.py` runs when the package is first imported.
- A module's top-level code runs on its first import in a Python process.

Explicit Import of Package Modules

- Dots in an import follow directories in the package.
- `player.timing` refers to `player/timing.py`.
- The full call is `package.module.function()`.

```
import player.timing

print(player.timing.format_duration(222))  # 3:42
```

demo06 · explicitimport

Importing Names from a Package

- A package can expose selected functions from its internal modules.
- `player/__init__.py` contains this import :

```
from player.timing import format_duration
```

- Callers can then use the shorter package-level name.

```
import player

print(player.format_duration(222))  # 3:42

from player import format_duration

print(format_duration(65))  # 1:05
```

Importing a Package

```
import player

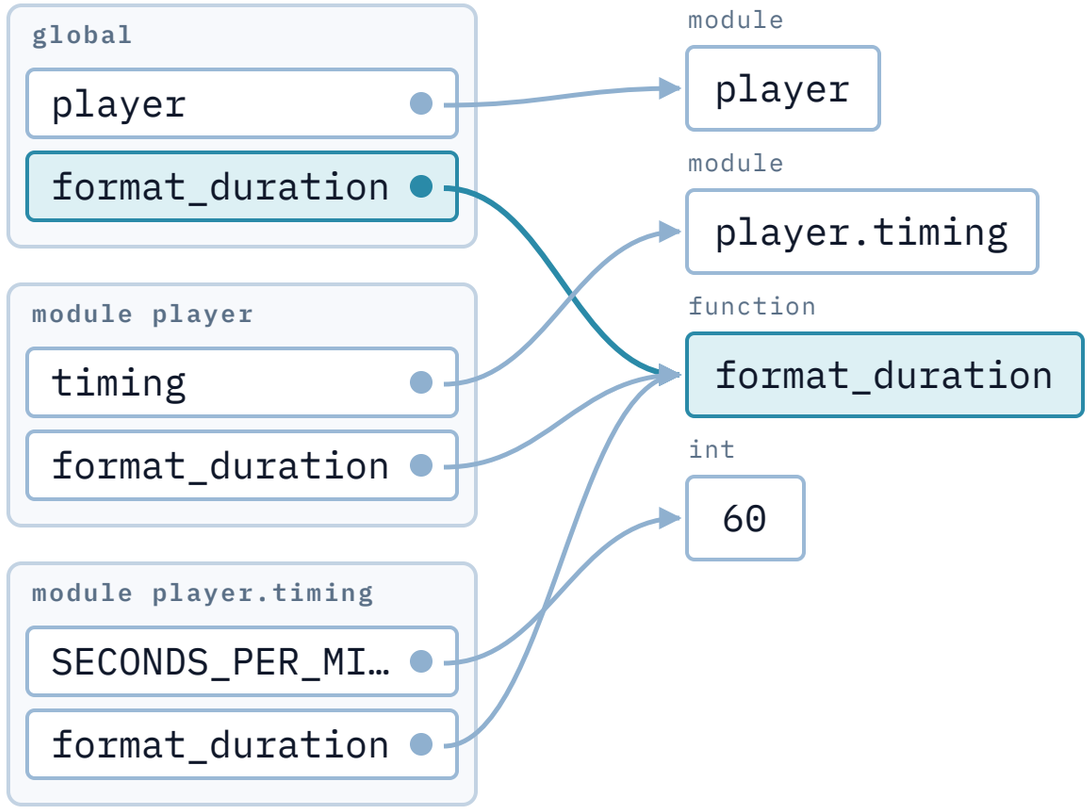
print(player.format_duration(222))  # 3:42

from player import format_duration

print(format_duration(65))  # 1:05
```

OUTPUT

```
3:42
1:05
```



Called without a prefix, the function returns `1:05`

demo07 · implicitimport

Running a Module Directly

- `__name__` is `"__main__"` when a file runs directly.
- When imported, `__name__` is the module's name.
- The guard runs `main()` only for direct execution.
- Importing defines `main` without printing the playlist display.

```
from timing import format_duration


def main() -> None:
    print(f"Duration: {format_duration(222)}")


if __name__ == "__main__":
    main()
```

SpiralTrain developer courses

Running Packages with python -m

- A package's `__main__.py` is its entry point.
- The following code belongs in `player/__main__.py`.
- Run from the directory containing `player/` :

```
uv run python -m player
```

```
from player.timing import format_duration


def main() -> None:
    print(f"Duration: {format_duration(222)}")


if __name__ == "__main__":
    main()
```

SpiralTrain developer courses

Installing Packages

- **Third-party packages are hosted on PyPI and installed with `uv add` :**
  - `uv add package` installs a package and records it in `pyproject.toml`
  - `uv add --dev package` adds a development tool, such as `pytest`
  - `uv remove package` uninstalls a dependency
- **Two files track dependencies and belong in version control :**
  - `pyproject.toml` declares the project's direct requirements
  - `uv.lock` records the exact resolved versions for reproducibility
- **Anyone with the project can reproduce the exact environment :**

```
uv add requests
uv add "requests>=2.28,<3"
uv add --dev pytest
uv sync          # install exactly what uv.lock says
uv remove requests
```

SpiralTrain developer courses

Virtual Environments

- **Virtual environments isolate dependencies per project :**
  - Each project has its own libraries and Python executable
  - Different projects can require different versions without conflict
- **`uv` manages virtual environments automatically :**
  - `.venv/` is created on the first `uv run` or `uv add`
  - Packages are installed into `.venv/` without touching system Python
- **There is no environment to activate manually :**
  - `uv run` puts the project's `.venv` in front for the command

```
myproject/
├── pyproject.toml   dependencies declared for this project
├── uv.lock          pinned versions of all installed packages
└── .venv/           created on the first uv run or uv add
```

- **Run code through `uv run`, which uses the project's interpreter :**

```
uv run python app.py
```

- **Without `uv`, environments are made by `python -m venv .venv` and activated by hand**

SpiralTrain developer courses

Summary : Modules

- **Modules organize code into reusable files :**
  - A module is a `.py` file defining functions, classes and variables
  - `import` and `from ... import` bring names into the calling file
- **Python locates modules using `sys.path` :**
  - Script directory (current directory for REPL or `-m`), then `PYTHONPATH` and defaults
  - `dir()` lists the names defined or imported in a module
- **Packages group modules into directories with `__init__.py` :**
  - Modules inside packages use dotted import paths
  - `python -m package` runs a package via its `__main__.py`
- **Executing versus importing :**
  - `if __name__ == "__main__":` guards code that only runs when executed directly
- **Dependency and environment management :**
  - `uv add` installs packages into an automatic `.venv`
  - `uv run` executes commands with the project's environment active

Comprehensions

Python Intro

Variables and Data Types

Data Structures

Control Flow

Functions

Modules

Comprehensions

Exception Handling

Python IO

Python Database Access

Python Classes

APPENDIX Type Hinting

APPENDIX Python Libraries

List Comprehensions

Syntax List Comprehensions

Filtered List Comprehension

Nested List Comprehensions

Flattening a List of Lists

Comprehension Features

Zip Function

Dictionary Construction with zip

Dictionary Comprehensions

Dictionary from Keys

Set Comprehensions

Generator Expressions

Built-ins that Take a Function

Recognising map and filter

Comprehensions and map

Recognising reduce

Summary : Comprehensions

List Comprehensions

- A comprehension builds a new list by evaluating an expression for each item.
- Read the `for` part first, then the expression on its left.

```
filenames: list[str] = ["cover", "stage", "crowd"]
photo_files = [f"{name}.jpg" for name in filenames]
print(photo_files)  # ['cover.jpg', 'stage.jpg', 'crowd.jpg']
```

demo01 · list_comprehensions

Syntax List Comprehensions

- A comprehension has the form `[expression for name in source]`.
- `source` supplies the items; `name` receives one item at a time.
- `expression` computes the value added to the new list.

```
filenames: list[str] = ["cover", "stage", "crowd"]
photo_files = []
for name in filenames:
    photo_files.append(f"{name}.jpg")
print(photo_files)  # ['cover.jpg', 'stage.jpg', 'crowd.jpg']

photo_files = [f"{name}.jpg" for name in filenames]
print(photo_files)  # ['cover.jpg', 'stage.jpg', 'crowd.jpg']
```

- Both forms create a new list and leave `filenames` unchanged.

SpiralTrain developer courses

Filtered List Comprehension

- An optional `if` keeps only the items that satisfy its condition.
- These two forms produce the same list :

```
filenames = ["cover.jpg", "notes.txt", "stage.jpg"]
photo_files = []
for name in filenames:
    if name.endswith(".jpg"):
        photo_files.append(name)
print(photo_files)  # ['cover.jpg', 'stage.jpg']

photo_files = [name for name in filenames if name.endswith(".jpg")]
print(photo_files)  # ['cover.jpg', 'stage.jpg']
```

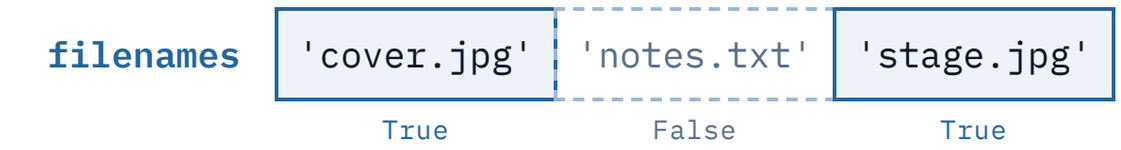
demo02 · comprehension_with_predicate

Filtering into a New List

```
filenames = ["cover.jpg", "notes.txt", "stage.jpg"]
photo_files = [name for name in filenames if name.endswith(".jpg")]
print(photo_files)  # ['cover.jpg', 'stage.jpg']
```

OUTPUT

```
['cover.jpg', 'stage.jpg']
```





The new list refers to the same two strings as `filenames`

demo02 · comprehension_with_predicate

Nested List Comprehensions

- A nested comprehension builds a list of lists, preserving the row structure.

```
photo_sizes = [[18.2, 19.6], [20.1, 21.8]]
rounded = [[round(value) for value in row] for row in photo_sizes]
print(rounded)  # [[18, 20], [20, 22]]
```

- Each row holds the photo sizes in megabytes from one album.
- The inner comprehension rounds the photo sizes in one row.
- The outer comprehension repeats that operation for every row.
- Use an ordinary loop when the nesting needs more explanation.

demo03 · nested_comprehensions

Flattening a List of Lists

- Two `for` clauses collect all the photo_sizes into one list.
- Their order is the same as in the equivalent nested loops.

```
photo_sizes = [[18.2, 19.6], [20.1, 21.8]]
flat = []
for row in photo_sizes:
    for value in row:
        flat.append(value)
print(flat)  # [18.2, 19.6, 20.1, 21.8]

flat = [value for row in photo_sizes for value in row]
print(flat)  # [18.2, 19.6, 20.1, 21.8]
```

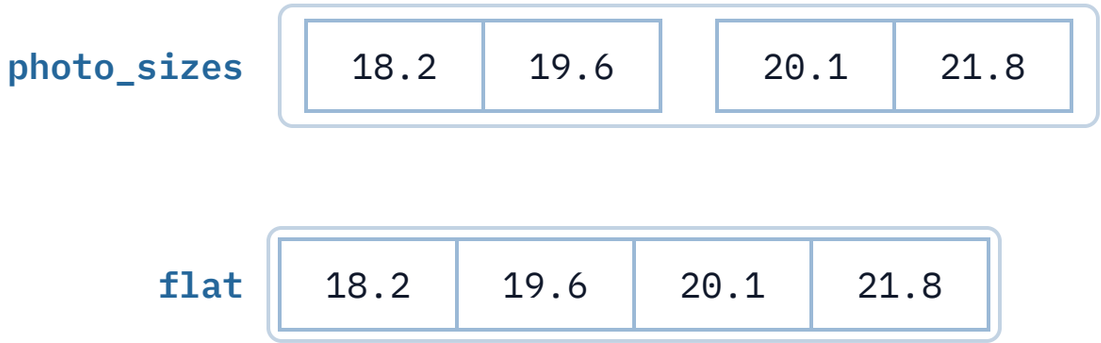
- The result can be used to calculate a minimum or maximum across all albums.

Two for Clauses, One List

```
photo_sizes = [[18.2, 19.6], [20.1, 21.8]]
flat = [value for row in photo_sizes for value in row]
print(flat)  # [18.2, 19.6, 20.1, 21.8]
```

OUTPUT

```
[18.2, 19.6, 20.1, 21.8]
```



The structure is gone : one list, four floats

SpiralTrain developer courses

Comprehension Features

- Unpack each track into a title and duration inside the `for` part
- The expression can call a function :

```
def format_duration(seconds: int) -> str:
    minutes = seconds // 60
    remainder = seconds % 60
    return f"{minutes}:{remainder:02d}"


tracks = [("Imagine", 183), ("Yesterday", 125), ("Respect", 149)]
labels = [f"{title} [{format_duration(seconds)}]" for title, seconds in tracks]
print(labels)
```

```
['Imagine [3:03]', 'Yesterday [2:05]', 'Respect [2:29]']
```

Zip Function

- `zip` pairs each track title with its duration :
  - It stops when the shortest input runs out by default
  - `strict=True` raises `ValueError` if their lengths differ

```
titles = ["Imagine", "Yesterday", "Respect"]
lengths = [183, 125, 149]
pairs = list(zip(titles, lengths))
print(pairs)
checked = list(zip(titles, lengths, strict=True))
print(checked)
```

```
[('Imagine', 183), ('Yesterday', 125), ('Respect', 149)]
[('Imagine', 183), ('Yesterday', 125), ('Respect', 149)]
```

SpiralTrain developer courses

Dictionary Construction with zip

- `dict` builds a lookup from pairs of keys and values :

```
titles = ["Imagine", "Yesterday", "Respect"]
lengths = [183, 125, 149]
durations = dict(zip(titles, lengths))
print(durations)
```

```
{'Imagine': 183, 'Yesterday': 125, 'Respect': 149}
```

demo06 · dictionary_with_zip

Dictionary Comprehensions

- `key: value` computes each dictionary entry
- An optional `if` keeps only matching tracks :

```
durations = {"Imagine": 183, "Yesterday": 125, "Respect": 149}
short_tracks = {title: seconds for title, seconds in durations.items() if seconds < 150}
print(short_tracks)  # {'Yesterday': 125, 'Respect': 149}
minutes = {title: round(seconds / 60, 2) for title, seconds in durations.items()}
print(minutes)  # {'Imagine': 3.05, 'Yesterday': 2.08, 'Respect': 2.48}
```

demo07 · dictionary_comprehension

Dictionary from Keys

- `dict.fromkeys` gives each key the same value
- A comprehension gives each playlist its own mutable track list :

```
albums = ["Night drive", "Warm-up"]
play_counts = dict.fromkeys(albums, 0)
print(play_counts)
playlists: dict[str, list[str]] = {name: [] for name in albums}
playlists["Night drive"].append("Imagine")
print(playlists)
```

```
{'Night drive': 0, 'Warm-up': 0}
{'Night drive': ['Imagine'], 'Warm-up': []}
```

- `dict.fromkeys(albums, [])` would make every album share one list

Set Comprehensions

- **A set comprehension keeps each result once :**
  - Titles that differ only in spacing and case can become one value

```
titles = [" Imagine ", "Yesterday", "imagine", "Respect "]
unique = {title.strip().lower() for title in titles}
print(sorted(unique))  # ['imagine', 'respect', 'yesterday']

files = ["cover.jpg", "notes.txt", "cover.jpg", "stage.jpg"]
unique_photos = {name for name in files if name.endswith(".jpg")}
print(sorted(unique_photos))  # ['cover.jpg', 'stage.jpg']
```

- **Sets have no display order; `sorted` makes printed output predictable**

Generator Expressions

- **A generator expression supplies values without building a new list :**
  - These photo sizes are checked and totalled as they are consumed

```
sizes_mb = [12, 8, 25]
print(sum(size for size in sizes_mb if size >= 10))  # 37
print(any(size > 20 for size in sizes_mb))  # True
print(all(size <= 20 for size in sizes_mb))  # False
print(any([]), all([]))  # False True
```

- `any` **and** `all` **stop as soon as the answer is known**
- **As the only argument of a call, the expression needs no extra parentheses**
- **Generator functions and** `yield` **belong to Advanced Python**

SpiralTrain developer courses

Built-ins that Take a Function

- A `key` function selects the value used for ordering :
  - Track titles can be sorted alphabetically or by title length

```
titles = ["Bohemian Rhapsody", "Yesterday", "Imagine"]
print(sorted(titles))  # ['Bohemian Rhapsody', 'Imagine', 'Yesterday']
print(sorted(titles, key=len))  # ['Imagine', 'Yesterday', 'Bohemian Rhapsody']
print(min(titles, key=len), max(titles, key=len))  # Imagine Bohemian Rhapsody
```

- `sorted` returns a new list; `list.sort` changes a list in place
- `min` and `max` also accept a key function

SpiralTrain developer courses

Recognising map and filter

- `map` transforms each item; `filter` keeps matching items
- Both return iterators; `list` collects the results :

```
def clean_filename(name: str) -> str:
    return name.strip().lower()


def is_photo(name: str) -> bool:
    return name.endswith(".jpg")


files = ["  COVER.JPG ", "STAGE.JPG", "NOTES.TXT"]
cleaned = list(map(clean_filename, files))
photos = list(filter(is_photo, cleaned))
print(photos)  # ['cover.jpg', 'stage.jpg']
print([name for name in cleaned if is_photo(name)])
```

- These forms also occur in existing code

demo12 · map_and_filter_functions

Comprehensions and map

- A comprehension can express the same filename transformation as `map` :

```
files = ["COVER.JPG", "STAGE.JPG"]
mapped = list(map(lambda name: name.lower(), files))
print(mapped)  # ['cover.jpg', 'stage.jpg']
comprehended = [name.lower() for name in files]
print(comprehended)  # ['cover.jpg', 'stage.jpg']
```

- Choose the form that makes the operation clear

SpiralTrain developer courses

Recognising reduce

- `functools.reduce` **combines items into one running result :**
  - Each step adds the next track's duration to the accumulated duration

```
from functools import reduce

lengths = [183, 125, 149]
total = reduce(lambda running, seconds: running + seconds, lengths)
print(total)  # 457
print(sum(lengths))  # 457
```

- **Prefer** `sum` **when it names the operation directly**

SpiralTrain developer courses

Summary : Comprehensions

- Build lists, dictionaries and sets with comprehensions.
- Use an optional `if` to filter the input.
- Keep nested comprehensions simple.
- Feed generator expressions into `sum`, `any` and `all`.
- Pass a `key` function to `sorted`, `min` and `max`.
- Recognise `map`, `filter` and `reduce` when reading other code.

Exception Handling

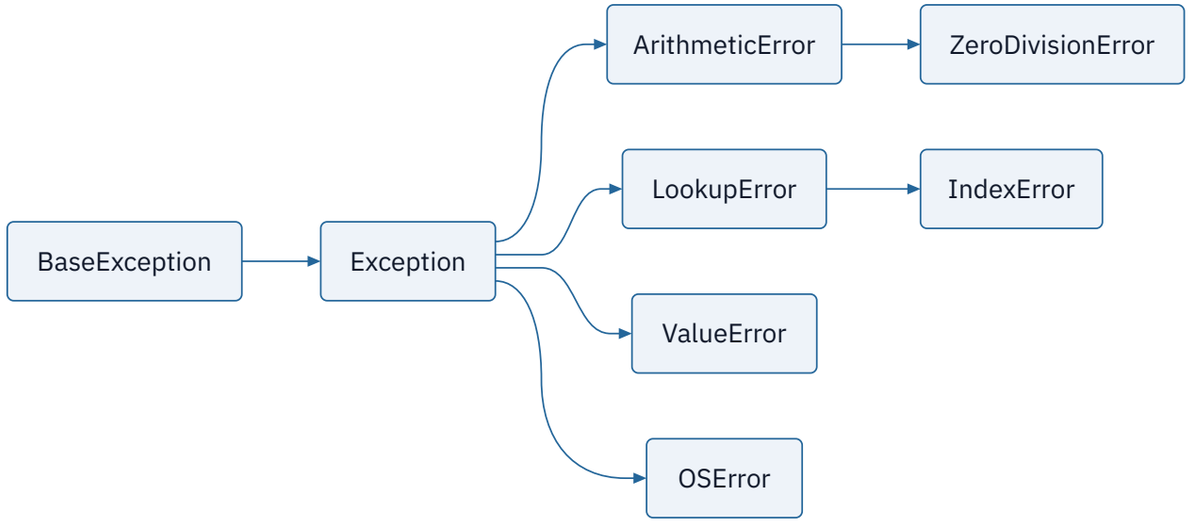
Python Intro
Variables and Data Types
Data Structures
Control Flow
Functions
Modules
Comprehensions
Exception Handling
Python IO
Python Database Access
Python Classes
APPENDIX Type Hinting
APPENDIX Python Libraries

Unexpected Errors
Reading a Traceback
Exception Handling
Typed Exception Handling
Exception Handling with Else
except Clause Multiple Exceptions
Python Standard Exceptions
Assertions Check Programmer Assumptions
try-finally Clause
Example try-finally
Exception Arguments
Example Argument of an Exception
Raising Exceptions
Example Raising an Exception
User Defined Exceptions

Summary : Exception Handling

Unexpected Errors

- **Python provides exceptions to handle unexpected errors :**
  - Exceptions are events disrupting normal program flow
  - May occur during the execution of program instructions
- **When Python encounters conditions it can't cope with :**
  - An exception is raised
  - An exception is a Python object that represents an error
- **Exception types are classes, and those classes form a tree :**



- **Catching a class catches everything below it in that tree**
- **When a Python script raises an exception :**
  - Python looks for a matching handler in the callers; if none exists, the script stops
- **Assertions check internal programmer assumptions :**
  - A false assertion raises `AssertionError`

Reading a Traceback

- A playlist contains a duration that cannot be converted to an integer :

```
def parse_seconds(text: str) -> int:
    return int(text)


parse_seconds("three")
```

```
Traceback (most recent call last):
  File "01_unexpectederrors.py", line 5, in <module>
    parse_seconds("three")
    ~~~~~~~~~~~~~^^^^^^^^^
  File "01_unexpectederrors.py", line 2, in parse_seconds
    return int(text)
ValueError: invalid literal for int() with base 10: 'three'
```

- Read the type and message, then the failing line and its caller
- File paths are shortened; line numbers refer to the demo

Exception Handling

- A `try` block contains an operation that may fail
- A matching `except` block handles the failure :

```
text = "three"
try:
    seconds = int(text)
except ValueError:
    print(f"Invalid track length: {text!r}")
```

```
Invalid track length: 'three'
```

- Catch a specific error that the program can handle
- A bare `except:` also catches keyboard interrupts; avoid it for ordinary input errors

Typed Exception Handling

- **Handlers are tested in order; only the first matching handler runs :**
  - A specific error belongs before a broader class

```
try:
    seconds = int("three")
except ValueError:
    print("Track length must be a whole number")
except Exception as error:
    print(f"Other problem: {error}")
else:
    print(f"Track length: {seconds}")
```

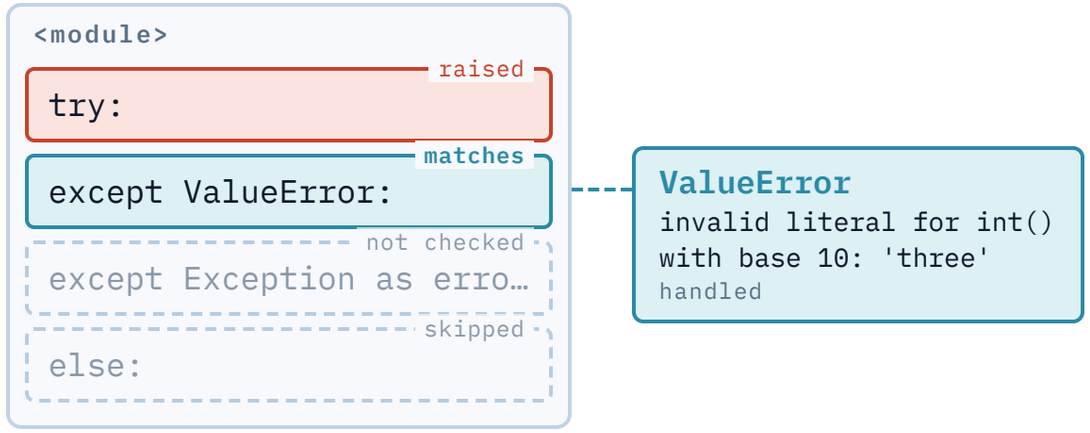
- **`ValueError` also belongs to `Exception`, so the second handler is not reached**
- **A broad handler is useful only when the program has a meaningful recovery**
- **Unmatched errors propagate to the caller**

The First Matching Handler

```
try:
    seconds = int("three")
except ValueError:
    print("Track length must be a whole number")
except Exception as error:
    print(f"Other problem: {error}")
else:
    print(f"Track length: {seconds}")
```

OUTPUT

```
Track length must be a whole number
```



Its block runs; `except Exception` would also match but is not checked, and `else` is skipped

Exception Handling with Else

- `else` **runs only when** `try` **finishes without an exception :**
  - Keep the conversion in `try` and the successful result in `else`

```
text = "222"
try:
    seconds = int(text)
except ValueError:
    print(f"Invalid track length: {text!r}")
else:
    print(f"Track length: {seconds} seconds")
```

```
Track length: 222 seconds
```

except Clause Multiple Exceptions

- A playlist duration may be missing or contain a nonnumeric label
- One handler can give both cases the same recovery :

```
lengths = [222, None, "live"]
for length in lengths:
    try:
        print(f"Length: {int(length)} seconds")
    except (TypeError, ValueError):
        print(f"Missing or invalid duration: {length!r}")
```

```
Length: 222 seconds
Missing or invalid duration: None
Missing or invalid duration: 'live'
```

- Catch a tuple of exception types when the same action handles each one
- Use separate handlers when the recovery differs

demo05 · multipleexceptions

Python Standard Exceptions

Exception	Typical cause
`ValueError`	A valid type has an unsuitable value, such as `int("six")`.
`TypeError`	An operation receives an unsuitable type.
`NameError`	A name has not been defined.
`AttributeError`	An object has no such attribute.
`IndexError`	A sequence index is out of range.
`KeyError`	A dictionary has no such key.
`ZeroDivisionError`	Division by zero.
`OSError`	A file or operating system operation fails.
`FileNotFoundError`	A requested file or directory does not exist.
`AssertionError`	An `assert` condition is false.
`EOFError`	`input()` reaches the end of the input, with Ctrl+D on Linux and macOS, or Ctrl+Z and Enter on Windows.
`KeyboardInterrupt`	The user presses Ctrl+C. It is a `BaseException`, so `except Exception` does not catch it.

Assertions Check Programmer Assumptions

- **An assertion checks an internal assumption about the program's own logic :**
  - Reversing a playlist must preserve its number of tracks

```
def reverse_playlist(tracks: list[str]) -> list[str]:
    reversed_tracks = tracks[::-1]
    assert len(reversed_tracks) == len(tracks), "Reversing must preserve every track"
    return reversed_tracks

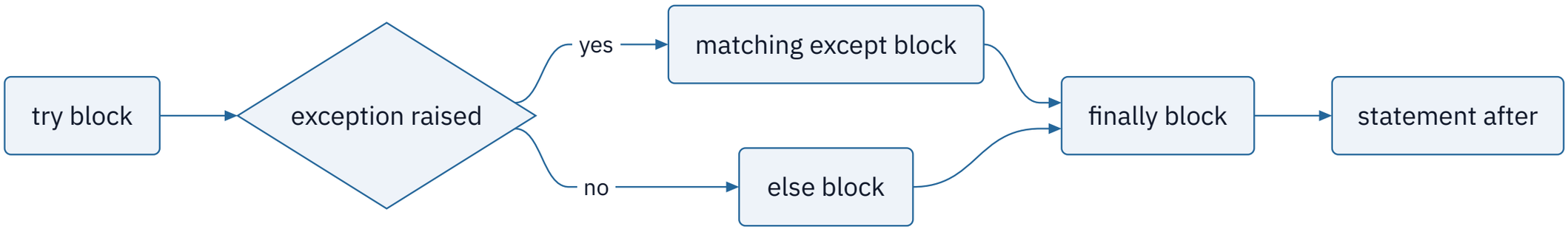

print(reverse_playlist(["Imagine", "Yesterday", "Respect"]))
```

- **An incorrect slice such as `tracks[:-1]` makes this assertion fail**
- **Assertions can be disabled with `-O`**
- **Required checks on incoming data use `raise`, so they remain active**

demo06 · assertionerror

try-finally Clause

- **`finally` runs when execution leaves a `try` block :**
  - It runs on success, on `return` and while an exception propagates
  - It performs cleanup; it does not handle the exception



- **An unmatched exception also passes through `finally` before reaching a caller**
- **`except`, `else` and `finally` can be combined in one statement**

Example try-finally

- An invalid track length leaves the function through its `finally` block :

```
def parse_seconds(text: str) -> int:
    try:
        return int(text)
    finally:
        print("Duration check finished")


try:
    parse_seconds("live")
except ValueError:
    print("Duration unavailable")
```

```
Duration check finished
Duration unavailable
```

- The caller handles the error after the final message appears

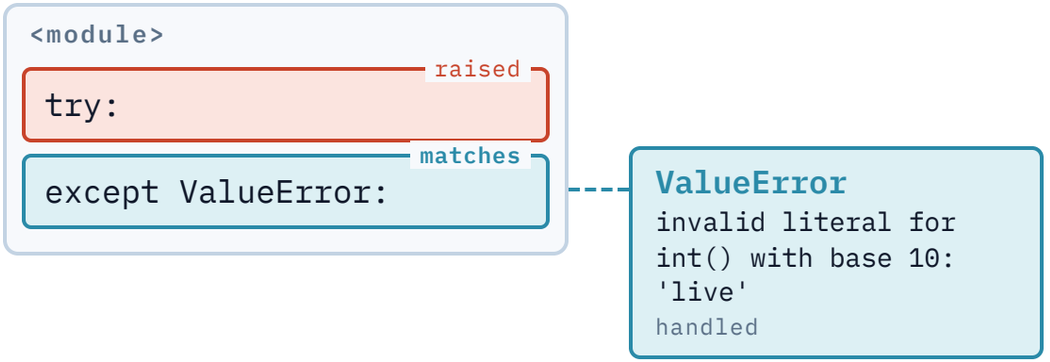
finally While an Exception Propagates

```python
def parse_seconds(text: str) -> int:
    try:
        return int(text)
    finally:
        print("Duration check finished")


try:
    parse_seconds("live")
except ValueError:
    print("Duration unavailable")
```

OUTPUT

```
Duration check finished
Duration unavailable
```



The handler runs; the program continues after the `try` statement

SpiralTrain developer courses

Exception Arguments

- `except ValueError as error` binds the exception object to `error`.
- `str(error)` gives its message; `print(error)` uses that text.
- `error.args` holds the constructor arguments as a tuple.
- Some types expose additional attributes, such as `OSError.filename`.

SpiralTrain developer courses

Example Argument of an Exception

- The exception retains the detail needed to report the faulty value :

```
text = "live"
try:
    int(text)
except ValueError as error:
    print(f"Cannot read track length {text!r}: {error}")
    print(error.args)
```

```
Cannot read track length 'live': invalid literal for int() with base 10: 'live'
("invalid literal for int() with base 10: 'live'",)
```

demo08 · exceptionarguments

Raising Exceptions

- `raise` reports a value that converts successfully but breaks a required rule :
  - A duration's seconds field must be between `0` and `59`

```
def parse_duration(text: str) -> int:
    minutes_text, seconds_text = text.split(":")
    minutes = int(minutes_text)
    seconds = int(seconds_text)
    if minutes < 0 or not 0 <= seconds < 60:
        raise ValueError(f"Invalid duration: {text!r}")
    return minutes * 60 + seconds
```

- The error message identifies the rejected duration
- `raise` inside an `except` block re-raises the current exception
- `raise NewError(...) from error` records another error as its cause

Example Raising an Exception

- The parser returns seconds for valid text and raises for an invalid duration :

```
def parse_duration(text: str) -> int:
    minutes_text, seconds_text = text.split(":")
    minutes = int(minutes_text)
    seconds = int(seconds_text)
    if minutes < 0 or not 0 <= seconds < 60:
        raise ValueError(f"Invalid duration: {text!r}")
    return minutes * 60 + seconds


print(parse_duration("3:42"))
try:
    parse_duration("3:99")
except ValueError as error:
    print(error)
    raise  # Let the caller decide how to recover.
```

- The first call prints `222`; the second reports `Invalid duration: '3:99'`
- The final `raise` propagates the error after it has been reported

User Defined Exceptions

- A specific exception type lets a caller distinguish a playlist rule failure :

```
class InvalidDurationError(ValueError):
    pass
```

- `InvalidDurationError` is also a `ValueError`; the existing broader handler still matches

demo10 · userdefinedexceptions

SpiralTrain developer courses

Summary : Exception Handling

- **Exceptions are events disrupting normal program flow :**
  - Python provides exception handling mechanism for unexpected errors
- **Code that might raise an exception is placed in a `try` block :**
  - This code is tested for exceptions
- **`try` block is followed by an `except` block :**
  - Control jumps to this block in case of exceptions and exceptions are handled there
- **`try` block may also be followed by a `finally` block :**
  - Is always executed whether or not the try-block raised an exception
- **Exceptions can have an argument :**
  - Value that gives additional information about the problem
- **Python has a number of predefined exceptions like :**
  - `ValueError`, `TypeError`, `KeyError` and `OSError`
- **Exceptions can be raised with the `raise` statement**
- **It is possible to create User Defined Exceptions :**
  - By deriving classes from standard built-in exceptions

Python IO

Python Intro
Variables and Data Types
Data Structures
Control Flow
Functions
Modules
Comprehensions
Exception Handling
Python IO
Python Database Access
Python Classes
APPENDIX Type Hinting
APPENDIX Python Libraries

Input and Output
Opening Files
File Open Modes
Using with
Reading Text Files
Writing Text Files
Closing Files
File Object Attributes
IO Module
Result of Calling open
Bytes and Byte Arrays
Reading Binary Files
Writing Binary Files
File Positions
The csv Module

Reading CSV Rows by Name
Paths with pathlib
Paths Relative to a Module
Renaming and Deleting Files
Creating and Removing Directories
Directory Contents
Summary : Python IO

SpiralTrain developer courses

Input and Output

- `print` writes its arguments to standard output, separated by spaces.
- `input` reads one line from standard input and removes the trailing newline.
- It returns a string; convert it before doing arithmetic.

```
print("Now playing:", "So What", "by", "Miles Davis")
volume = int(input("Volume (0-100) : "))
print(f"Volume set to {volume}%")
```

```
Now playing: So What by Miles Davis
Volume (0-100) : 65
Volume set to 65%
```

SpiralTrain developer courses

Opening Files

- `open` **function :**
  - Opens a file to be able to read or write a file later
  - Creates file object which can be used to call other support methods

```
file_object = open(file, mode="r", buffering=-1, encoding=None)
```

- `file` **:**
  - String value that contains the name of the file to be accessed
- `mode` **:**
  - Determines mode in which the file has to be opened, read, write, append etc.
  - `t` for text mode is default and optional, `b` is for byte mode
- `encoding` **names the encoding of a text file, such as** `"utf-8"`
- `buffering` **:**
  - `0` disables buffering, but is allowed only in binary mode
  - `1` enables line buffering in text mode
  - If buffering > 1 : Buffering performed with indicated buffer size

SpiralTrain developer courses

File Open Modes

- **Python distinguishes between binary and text I/O :**
  - In binary mode content is returned as bytes objects without any decoding
  - In text mode, as `str`, bytes decoded using given or platform-specific encoding

Mode	Description
`r` / `rb`	Reading only, text or binary format. Default is text mode
`r+` / `rb+`	Both reading and writing, text or binary format
`w` / `wb`	Writing only, text or binary. Overwrites existing file or creates new file
`w+` / `wb+`	Both writing and reading, text or binary. Overwrites or creates file
`a` / `ab`	Appending, text or binary. File pointer is at the end of the file
`a+` / `ab+`	Both appending and reading, text or binary format

SpiralTrain developer courses

Using with

- `with` closes the file when the block ends, even after an exception or `return`.
- Closing writes any buffered text to the file.
- Prefer this form whenever you open a file.

```
with open("last_played.txt", mode="w", encoding="utf-8") as fw:
    fw.write("So What\n")
# The file is closed here, even if write() raised.
print(fw.closed)  # True

with open("last_played.txt", encoding="utf-8") as fr:
    print(fr.read(), end="")  # So What
```

with Closes the File

```python
with open("last_played.txt", mode="w", encoding="utf-8") as fw:
    fw.write("So What\n")
# The file is closed here, even if write() raised.
print(fw.closed)  # True

with open("last_played.txt", encoding="utf-8") as fr:
    print(fr.read(), end="")  # So What
```

last_played.txt

on disk · last_played.txt

S	o		W	h	a	t	\n

OUTPUT

```
True
So What
```

fw — mode 'w' — closed

fr — mode 'r' — closed

The second block ends and closes `fr`; the title stays in `last_played.txt` for the next run

SpiralTrain developer courses

Reading Text Files

- `readline()` reads one line, including its `\n`.
- `read()` reads from the current position to the end of the file :
  - `read(count)` reads at most `count` characters in text mode
- A text file is also an iterable of its lines :

```
with open("playlist.txt", encoding="utf-8") as playlist:
    for number, line in enumerate(playlist, start=1):
        print(f"{number}. {line.rstrip()}")
```

```
1. So What
2. Blue in Green
3. Take Five
4. Jóga
```

- `rstrip()` removes the newline that ends each line.

SpiralTrain developer courses

Writing Text Files

- `write` writes a string to a file opened for writing :
  - It does not add a newline character (`\n`) to the end of the string
- Mode `w` creates the file or empties an existing one; mode `a` appends.
- `encoding="utf-8"` stores titles such as `Jóga` the same way on every platform.

```
with open("road_trip.txt", mode="w", encoding="utf-8") as fw:
    fw.write("Jóga\n")
    fw.write("Hoppípolla\n")
    fw.write("Svefn-g-englar\n")

with open("road_trip.txt", mode="a", encoding="utf-8") as fa:
    fa.write("Glósóli\n")
```

- File `road_trip.txt` then holds :

```
Jóga
Hoppípolla
Svefn-g-englar
Glósóli
```

demo04 · writetextfile

Closing Files

- **Without `with`, a file stays open until `close` is called :**
  - `close` writes unwritten data, then closes the file object
  - No more reading or writing can be done after calling `close`
  - Do not rely on garbage collection to close a file promptly
- **Older code opens and closes files by hand :**

```
playlist = open("playlist.txt", encoding="utf-8")
first = playlist.readline()
playlist.close()
print(first, end="")  # So What
```

- **An exception between `open` and `close` skips the `close`; `with` does not.**

SpiralTrain developer courses

File Object Attributes

- File objects expose their name, mode, encoding and closed state.

```
playlist = open("playlist.txt", encoding="utf-8")
print("Name of the file :", playlist.name)
print("Opening mode     :", playlist.mode)
print("Encoding         :", playlist.encoding)
print("First line       :", playlist.readline(), end="")
print("Closed or not    :", playlist.closed)
playlist.close()
print("Closed or not    :", playlist.closed)
```

```
Name of the file : playlist.txt
Opening mode     : r
Encoding         : utf-8
First line       : So What
Closed or not    : False
Closed or not    : True
```

IO Module

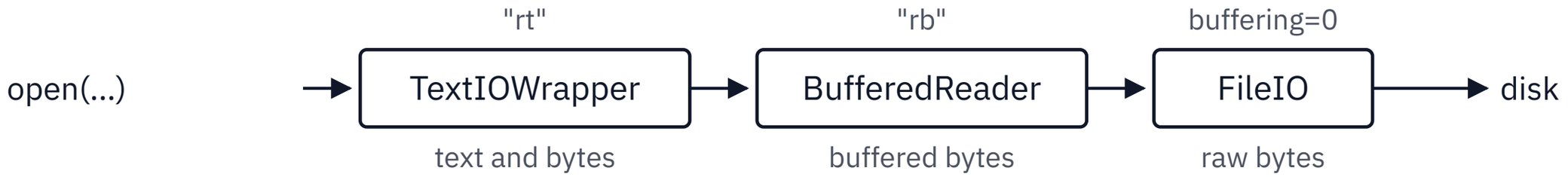
- Core of the I/O system is implemented in io library module
- Consists of a collection of different I/O classes :
  - `FileIO`
  - `BufferedReader`
  - `BufferedWriter`
  - `BufferedRWPair`
  - `BufferedRandom`
  - `TextIOWrapper`
  - `BytesIO`
  - `StringIO`
- Each class implements a different kind of I/O
- Classes get layered to add features

Result of Calling open

- The object returned varies with the file mode and the buffering :

```
>>> open("playlist.txt", "rt", encoding="utf-8")
<_io.TextIOWrapper name='playlist.txt' mode='rt' encoding='utf-8'>
>>> open("playlist.txt", "rb")
<_io.BufferedReader name='playlist.txt'>
>>> open("playlist.txt", "rb", buffering=0)
<_io.FileIO name='playlist.txt' mode='rb' closefd=True>
```

- Opening a text file stacks three objects, and you are handed the outermost :



SpiralTrain developer courses

Bytes and Byte Arrays

- `bytes` is immutable; `bytearray` allows individual bytes to change.
- Encoding converts text to bytes; decoding converts bytes back to text :
  - In UTF-8, `ó` takes two bytes, so `Jóga` is four characters and five bytes
- Indexing returns an integer from 0 to 255, not a character.

```
title = "Jóga"
data = title.encode("utf-8")
print(data)  # b'J\xc3\xb3ga'
print(len(title), len(data))  # 4 5
print(data[0])  # 74
print(data.decode("utf-8"))  # Jóga

mutable = bytearray(data)
mutable[0] = 89
print(mutable)  # bytearray(b'Y\xc3\xb3ga')
print(mutable.decode("utf-8"))  # Yóga
```

- Binary files read and write bytes; text files handle encoding for you.

SpiralTrain developer courses

Reading Binary Files

- Binary mode (`rb`) reads bytes without decoding them.
- `read()` reads the remaining contents; `read(3)` reads up to three bytes.
- Binary mode takes no `encoding` argument.

```
with open("trilobyte.jpg", mode="rb") as photo:
    data = photo.read()
print(type(data))  # <class 'bytes'>
print(len(data))  # 49278, the size in bytes
print(data[:3].hex())  # ffd8ff, the start of every JPEG photo
```

- `hex()` shows each byte as two hexadecimal digits.

Writing Binary Files

- `wb` writes bytes and replaces any existing file.
- `struct` packs numbers into bytes and unpacks them with the same format.
- Here `<iii` means three four-byte integers in little-endian order.

```
from struct import pack, unpack

state = pack("<iii", 7, 184, 65)  # track number, seconds played, volume
with open("player.bin", mode="wb") as fw:
    fw.write(state)

with open("player.bin", mode="rb") as fr:
    track, seconds, volume = unpack("<iii", fr.read())
print(track, seconds, volume)  # 7 184 65
```

- `player.bin` holds twelve bytes, which a later run reads back to resume playback.

File Positions

- `tell()` reports the current position; `seek(offset)` moves it.
- Binary file positions count bytes from the beginning.
- Text file positions are opaque values; use `seek(0)` or a value from `tell()`.

```
with open("trilobyte.jpg", "rb") as photo:
    print(photo.read(2).hex())  # ffd8
    print(photo.tell())  # 2
    photo.seek(6)
    print(photo.read(4))  # b'JFIF'
    photo.seek(0)
    print(photo.read(2).hex())  # ffd8
```

- A JPEG photo in the JFIF format has the bytes `JFIF` from position 6.

Moving the File Position

```
with open("trilobyte.jpg", "rb") as photo:
    print(photo.read(2).hex())  # ffd8
    print(photo.tell())  # 2
    photo.seek(6)
    print(photo.read(4))  # b'JFIF'
    photo.seek(0)
    print(photo.read(2).hex())  # ffd8
```

trilobyte.jpg

on disk · trilobyte.jpg

ff	d8	ff	e0	00	10	J	F	I	F	...

photo
mode 'rb'
closed

OUTPUT

```
ffd8
2
b'JFIF'
ffd8
```

The block ends and closes `photo`; the photo itself is unchanged

The csv Module

- Use `csv` for tabular text; it handles quoted commas and embedded newlines.
- Open CSV files with `newline=""` so `csv` controls newline handling.

```
import csv

with open("history.csv", "w", newline="", encoding="utf-8") as fw:
    writer = csv.writer(fw)
    writer.writerow(["track", "artist", "seconds"])
    writer.writerows([["So What", "Miles Davis", 562], ["Hello, Goodbye", "The Beatles", 208]])

with open("history.csv", newline="", encoding="utf-8") as fr:
    for row in csv.reader(fr):
        print(row)
```

```
['track', 'artist', 'seconds']
['So What', 'Miles Davis', '562']
['Hello, Goodbye', 'The Beatles', '208']
```

Reading CSV Rows by Name

- `DictReader` uses the first row as column names.
- Field values are strings; convert numeric fields explicitly.
- `listening_history.csv` holds one row per track played :

```
total = 0
with open("listening_history.csv", newline="", encoding="utf-8") as fr:
    for row in csv.DictReader(fr):
        seconds = int(row["seconds"])
        total += seconds
        print(f"{row['track']}: {seconds} s")
print(f"Total: {total // 60} min {total % 60} s")
```

```
So What: 562 s
Hello, Goodbye: 208 s
Jóga: 305 s
Take Five: 324 s
Total: 23 min 19 s
```

SpiralTrain developer courses

Paths with pathlib

- **The `pathlib` module represents a path as an object, not a string :**
  - `Path` knows how to join, inspect, read, write, rename and delete
  - It is the modern replacement for stringly typed `os.path` work

```
from pathlib import Path

here = Path.cwd()  # the current working directory
song = here / "music" / "jazz" / "so_what.mp3"  # joining, with / instead of +
```

- **The `/` operator joins path segments :**
  - It produces the right separator on Windows, macOS and Linux alike
  - No more `"music" + "\\" + name`, and no more forgetting the separator

- **A path answers questions about itself :**

```
song.name      # so_what.mp3
song.stem      # so_what
song.suffix    # .mp3
song.parent    # the jazz directory, as a Path
song.exists()  # True or False
```

Paths Relative to a Module

- `Path(__file__)` **is the path of the running module, so** `parent` **is its folder :**
  - `resolve()` makes the path absolute, and follows any symbolic links
- `path.open()` **works like** `open()`**, and takes the same arguments :**

```
from pathlib import Path

here = Path(__file__).resolve().parent  # the folder of this module
print(here.is_absolute())  # True

with Path(__file__).open(encoding="utf-8") as source:
    print(source.readline().strip())  # from pathlib import Path
```

- `here / "playlist.txt"` **then finds a data file beside the module, whatever the working directory.**

Renaming and Deleting Files

- A `Path` renames and deletes itself :

```
from pathlib import Path

photo = Path("IMG_0412.jpg")
photo.touch()
renamed = photo.rename("2026-07-14 beach.jpg")
renamed.unlink()
```

- `touch` creates an empty file, `unlink` removes one :
  - `unlink(missing_ok=True)` does nothing when the file is already gone
- Short reads and writes need no `open` at all :

```
caption = Path("beach.txt")
caption.write_text("Sunset at the beach\n", encoding="utf-8")
print(caption.read_text(encoding="utf-8"), end="")
```

- The `os` module still has `os.rename` and `os.remove` :
  - They take strings and are what older code uses

SpiralTrain developer courses

Creating and Removing Directories

- `mkdir` creates a directory, `rmdir` removes an empty one :

```
from pathlib import Path

Path("albums").mkdir(exist_ok=True)
Path("albums").rmdir()
```

- Two keyword arguments cover the awkward cases :
  - `parents=True` creates every missing directory in the path
  - `exist_ok=True` succeeds quietly when the directory is already there

```
Path("albums/2026/07 beach").mkdir(parents=True, exist_ok=True)
```

- `rmdir` only removes an empty directory :
  - Remove the contents first, from the leaf upwards
  - `shutil.rmtree` removes a whole tree at once, and does not ask twice

SpiralTrain developer courses

Directory Contents

- `iterdir` lists direct entries; `glob` selects matching names.
- Each entry is a `Path` that you can inspect or open.
- `rglob` searches subdirectories too.

```
from pathlib import Path

here = Path.cwd()
album = here / "holiday"
album.mkdir(exist_ok=True)
for name in ["beach.jpg", "sunset.jpg", "notes.txt"]:
    (album / name).touch()

print("contents   :", sorted(p.name for p in album.iterdir()))
print("only photos:", sorted(p.name for p in album.glob("*.jpg")))
```

```
contents   : ['beach.jpg', 'notes.txt', 'sunset.jpg']
only photos: ['beach.jpg', 'sunset.jpg']
```

SpiralTrain developer courses

Summary : Python IO

- **Simplest way to produce output is to use the `print` function :**
  - Allows you to pass zero or more expressions, separated by commas
- **The `input` function reads a line of text from standard input :**
  - It always returns a string, so convert it before doing arithmetic with it
- **Open files with `with`, which closes them whatever happens :**
  - The manual `open` and `close` pair is shown so that you recognise older code
- **A text file is read whole, by line, or by iterating over it**
- **Files can be opened in several modes :**
  - For reading, for writing, for appending or combinations thereof
- **Binary files contain bytes; text files encode and decode strings.**
- **`tell` and `seek` report and move the file position**
- **Use `csv.reader`, `csv.writer` and `DictReader` for tabular text.**
- **`pathlib` represents a path as an object :**
  - `Path` joins with `/`, and creates, renames, deletes, reads and lists

Python Database Access

Python Intro

Variables and Data Types

Data Structures

Control Flow

Functions

Modules

Comprehensions

Exception Handling

Python IO

Python Database Access

Python Classes

APPENDIX Type Hinting

APPENDIX Python Libraries

Python DB API

DB API Concepts

Check Database Version

Using with

Inserting Data

Prepared Statements

Last inserted row id

Retrieving Data

Filtering and Grouping Data

Rows by Column Name

Fetching Rows

Parameterized Queries

Metadata

Transactions

Committing Data

Summary : Python Database Access

SpiralTrain developer courses

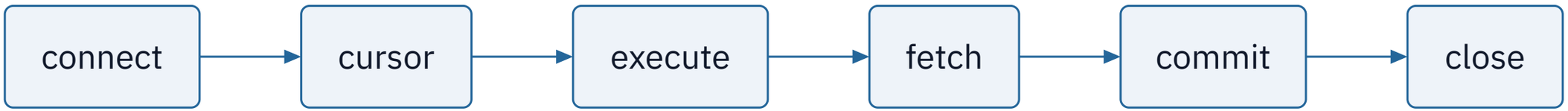
Python DB API

- **Provides a standard interface from Python to databases :**
  - Most Python database interfaces adhere to this standard
  - Each database has its own module highly consistent with others
- **The standard is described in PEP 249 :**
  - Modules are available for most of the popular relational databases
  - Code written for one database reads almost the same for another
- **Python DB API 2.0 compatible modules :**
  - `sqlite3` (SQLite), part of the standard library and used in this module
  - `psycopg` (PostgreSQL)
  - `mysqlclient` or `PyMySQL` (MySQL and MariaDB)
  - `oracledb` (Oracle)
  - `pyodbc` (any database with an ODBC driver, such as SQL Server and DB2)

SpiralTrain developer courses

DB API Concepts

- **DB API uses two main concepts for processing queries :**
  - Connections and cursors
- **Every script follows the same shape :**



- **Connection objects logically wrap a database connection :**
  - Provide access to the database and handle transactions
- `sqlite3.connect()` **opens a database file, and creates it if it does not exist**
- **Cursor objects used to manipulate and query the database :**
  - Created on connections using the `conn.cursor()` method
  - Cursor objects provide ways to execute statements and access results

```
con = sqlite3.connect("watchlist.db")
cur = con.cursor()
cur.execute("CREATE TABLE IF NOT EXISTS films(id INTEGER PRIMARY KEY, title TEXT)")
```

Check Database Version

- **A first query asks SQLite for its version :**
  - `fetchone()` returns the result row as a tuple, here with one value
  - `close()` releases the database file

```
import sqlite3

con = sqlite3.connect("watchlist.db")  # creates watchlist.db if it does not exist
cur = con.cursor()
cur.execute("SELECT sqlite_version()")
print(f"SQLite version: {cur.fetchone()[0]}")
con.close()
```

```
SQLite version: 3.50.4
```

- **The version depends on the Python installation.**

Using with

- **A connection used in a `with` statement manages the transaction :**
  - When the block ends normally, the changes are committed
  - When the block ends with an exception, the changes are rolled back

```
import sqlite3
con = sqlite3.connect("watchlist.db")
with con:
    cur = con.cursor()
    cur.execute("SELECT COUNT(*) FROM films")
con.close()  # `with con:` commits or rolls back, it does not close
```

- **The connection stays open after the block :**
  - Close it yourself with `con.close()` when you are done
- **`closing()` from `contextlib` calls `close()` when the block ends, so one `with` line can commit and close :**

```
from contextlib import closing

with closing(sqlite3.connect("watchlist.db")) as con, con:
    cur = con.cursor()
    cur.execute("SELECT COUNT(*) FROM films")
```

Inserting Data

- **Cursor objects provides a `cursor.execute()` method :**
  - Execute SQL statements on a connection
- **`INTEGER PRIMARY KEY` makes SQLite number the rows :**
  - An `INSERT` that leaves out `id` gets the next number
- **This demo creates the `films` table that the later demos read :**

```
import sqlite3
con = sqlite3.connect("watchlist.db")
with con:
    cur = con.cursor()
    cur.execute("DROP TABLE IF EXISTS films")
    cur.execute("""
        CREATE TABLE films(id INTEGER PRIMARY KEY, title TEXT, year INT,
                           minutes INT, watched INT DEFAULT 0, rating INT)
        """)
    cur.execute("INSERT INTO films(title, year, minutes) VALUES('Spirited Away', 2001, 125)")
    cur.execute("INSERT INTO films(title, year, minutes) VALUES('Amélie', 2001, 122)")
    cur.execute("INSERT INTO films(title, year, minutes) VALUES('Arrival', 2016, 116)")
    cur.execute("INSERT INTO films(title, year, minutes) VALUES('Paddington 2', 2017, 104)")
    cur.execute("INSERT INTO films(title, year, minutes) VALUES('Coco', 2017, 105)")
    cur.execute("INSERT INTO films(title, year, minutes) VALUES('Parasite', 2019, 132)")
```

SpiralTrain developer courses

Prepared Statements

- `executemany()` repeats one parameterized statement for each tuple
- Each `?` placeholder receives a value from that tuple, in order :

```
import sqlite3
films = (
    ("Spirited Away", 2001, 125),
    ("Amélie", 2001, 122),
    ("Arrival", 2016, 116),
    ("Paddington 2", 2017, 104),
    ("Coco", 2017, 105),
    ("Parasite", 2019, 132),
)
con = sqlite3.connect("watchlist.db")
with con:
    cur = con.cursor()
    cur.execute("DROP TABLE IF EXISTS films")
    cur.execute("""
        CREATE TABLE films(id INTEGER PRIMARY KEY, title TEXT, year INT,
                           minutes INT, watched INT DEFAULT 0, rating INT)
        """)
    cur.executemany("INSERT INTO films(title, year, minutes) VALUES(?, ?, ?)", films)
```

Last inserted row id

- **Sometimes we need to determine the id of last inserted row :**
  - In Python SQLite we use the `lastrowid` attribute of the cursor object
- **The name `":memory:"` opens a database that lives only while the program runs :**

```
import sqlite3
con = sqlite3.connect(":memory:")
with con:
    cur = con.cursor()
    cur.execute("CREATE TABLE films(id INTEGER PRIMARY KEY, title TEXT)")
    cur.execute("INSERT INTO films(title) VALUES('Spirited Away')")
    cur.execute("INSERT INTO films(title) VALUES('Arrival')")
    cur.execute("INSERT INTO films(title) VALUES('Paddington 2')")
    film_id = cur.lastrowid
    print(f"Paddington 2 was added with id {film_id}")
```

```
Paddington 2 was added with id 3
```

Inserts Wait for the Commit

```
import sqlite3
con = sqlite3.connect(":memory:")
with con:
    cur = con.cursor()
    cur.execute("CREATE TABLE films(id INTEGER PRIMARY KEY, title TEXT)")
    cur.execute("INSERT INTO films(title) VALUES('Spirited Away')")
    cur.execute("INSERT INTO films(title) VALUES('Arrival')")
    cur.execute("INSERT INTO films(title) VALUES('Paddington 2')")
    film_id = cur.lastrowid
    print(f"Paddington 2 was added with id {film_id}")
```

con	Connection, :memory:
cur	Cursor, lastrowid 3
film_id	3

table films

id	title
1	'Spirited Away'
2	'Arrival'
3	'Paddington 2'

OUTPUT

```
Paddington 2 was added
with id 3
```

The block ends without an exception, so `with con` commits : the three films are saved

Retrieving Data

- `fetchall()` **returns all rows of the result as a list of tuples :**
  - The `films` table comes from demo03 or demo04, so run one of them first

```
import sqlite3
con = sqlite3.connect("watchlist.db")
with con:
    cur = con.cursor()
    cur.execute("SELECT id, title, year FROM films")
    rows = cur.fetchall()
    for row in rows:
        print(row)
```

```
(1, 'Spirited Away', 2001)
(2, 'Amélie', 2001)
(3, 'Arrival', 2016)
```

- **Other fetch methods read part of the result :**

```
first_row = cur.fetchone()
next_10_rows = cur.fetchmany(10)
all_remaining_rows = cur.fetchall()
```

Filtering and Grouping Data

- `WHERE` filters rows; `?` binds the value from the parameter tuple
- `GROUP BY` forms groups; `COUNT(*)` counts rows and `SUM` adds non-NULL values
- `COALESCE(value, 0)` replaces SQL `NULL` with zero, including an empty `SUM`
- `ORDER BY` sorts results; `DESC` puts the largest total first

```
import sqlite3

# Run demo03 or demo04 first to create the films table.
con = sqlite3.connect("watchlist.db")
try:
    cur = con.cursor()
    cur.execute("""
        SELECT year, COUNT(*), COALESCE(SUM(minutes), 0)
        FROM films WHERE year >= ?
        GROUP BY year ORDER BY SUM(minutes) DESC
        """, (2010,))
    for row in cur.fetchall():
        print(row)
finally:
    con.close()
```

SpiralTrain developer courses

Rows by Column Name

- **A row is a tuple by default, so a column is read by its position :**
  - `row[1]` works, but the reader has to know what the second column is
- **Set `row_factory` on the connection to `sqlite3.Row` :**
  - Every row can then be read by column name as well as by position

```
import sqlite3
con = sqlite3.connect("watchlist.db")
con.row_factory = sqlite3.Row  # rows can now be read by column name
with con:
    cur = con.cursor()
    cur.execute("SELECT * FROM films")
    rows = cur.fetchall()
    for row in rows:
        print(f"{row['title']} ({row['year']}), {row['minutes']} min")
```

```
Spirited Away (2001), 125 min
Amélie (2001), 122 min
```

Fetching Rows

- **Returning all data at a time may not be feasible :**
  - It is also possible to fetch rows one by one with `fetchone()` :

```
import sqlite3
con = sqlite3.connect("watchlist.db")
with con:
    cur = con.cursor()
    cur.execute("SELECT id, title, minutes FROM films")
    while True:
        row = cur.fetchone()
        if row is None:
            break
        print(row[0], row[1], row[2])
```

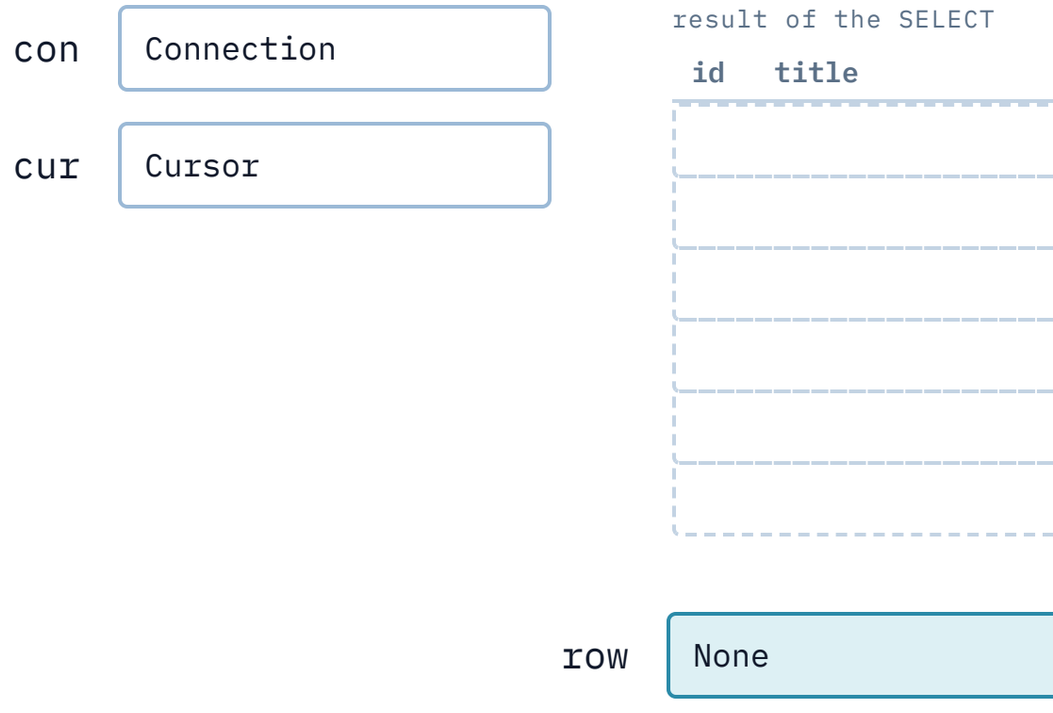
- **`fetchone()` method returns next row from the table :**
  - If there is no more data left, it returns `None`

One Row per fetchone

```
import sqlite3
con = sqlite3.connect("watchlist.db")
with con:
    cur = con.cursor()
    cur.execute("SELECT id, title, minutes FROM films")
    while True:
        row = cur.fetchone()
        if row is None:
            break
        print(row[0], row[1], row[2])
```

OUTPUT

```
1 Spirited Away 125
2 Amélie 122
3 Arrival 116
4 Paddington 2 104
5 Coco 105
6 Parasite 132
```



`row is None`, so `break` leaves the loop

SpiralTrain developer courses

Parameterized Queries

- **Passing data from Python to the database :**
  - First discouraged option is quoting values and sending plain SQL to the database
  - Second option preferred using binding parameters in the SQL with place holders
- **A bound value needs no quoting, so a title such as `Howl's Moving Castle` is safe**
- **Python sqlite3 module supports two types of placeholders :**
  - Question marks and named placeholders

```
import sqlite3
film_id = 3
rating = 5
con = sqlite3.connect("watchlist.db")
with con:
    cur = con.cursor()
    cur.execute("UPDATE films SET watched = 1, rating = ? WHERE id = ?", (rating, film_id))
    print(f"Number of rows updated: {cur.rowcount}")
```

- **Named placeholders take a dictionary :**

```
cur.execute("SELECT id, title FROM films WHERE title = :title", {"title": title})
```

SpiralTrain developer courses

Metadata

- **Metadata is information about the data in the database :**
  - Contains information about the tables and columns like name and datatypes

```
import sqlite3
con = sqlite3.connect("watchlist.db")
with con:
    cur = con.cursor()
    cur.execute("PRAGMA table_info(films)")
    data = cur.fetchall()
    for d in data:
        print(d[0], d[1], d[2])
```

```
0 id INTEGER
1 title TEXT
2 year INT
3 minutes INT
4 watched INT
5 rating INT
```

- **Metadata in SQLite can be obtained using `PRAGMA` command :**
  - Or by querying the SQLite system table `sqlite_master`

SpiralTrain developer courses

Transactions

- **Transactions logically wrap statements into blocks of execution :**
  - Transactions are units of recovery
- **Main benefit is that you can undo changes very easily :**
  - Ensures data consistency at all times
  - Not all databases provide transactions !
- **Transactions conform to ACID principles :**
  - Atomicity, Consistency, Isolation, Durability
- **DB API supports transactions if the database supports them :**
  - Via methods on the connection object
- **Write all changes of the last transaction block to the database :**

```
con.commit()
```

- **Undo all changes applied to the database on the connection :**

```
con.rollback()
```

Committing Data

- `save_ratings` stores a batch of ratings, all of them or none :

```
def save_ratings(ratings: list[tuple[int, int]]) -> None:
    con = sqlite3.connect("watchlist.db")
    try:
        cur = con.cursor()
        for film_id, stars in ratings:
            if not 1 <= stars <= 5:
                raise ValueError(f"rating must be 1 to 5, got {stars}")
            cur.execute("UPDATE films SET watched = 1, rating = ? WHERE id = ?", (stars, film_id))
        con.commit()  # without this line, close() discards the updates
    except (sqlite3.Error, ValueError) as e:
        con.rollback()  # undoes every update of this batch
        print(f"Error: {e}")
    finally:
        con.close()

save_ratings([(2, 3), (6, 9)])  # 9 is refused, so the rating of film 2 is undone too
save_ratings([(1, 4), (6, 5)])
```

```
Error: rating must be 1 to 5, got 9
```

- Afterwards films 1 and 6 are watched and rated; film 2 is not

SpiralTrain developer courses

Summary : Python Database Access

- **Python DB API is standard interface from Python to databases :**
  - Each database has its own module highly consistent with others
- **DB API uses two main concepts for processing queries :**
  - Connections and cursors
- **Connection objects logically wrap a database connection :**
  - Provide means to handle database transactions
- **Cursor objects used to manipulate and query the database :**
  - Created on connections using the `conn.cursor()` method
- **If same query must be repeated multiple times :**
  - Can use parameterized queries with question marks or named parameters
- **A connection in a `with` statement commits or rolls back the transaction :**
  - It does not close the connection; `con.close()` does that
- **DB API supports transactions if database supports them :**
  - `commit()` keeps a batch of changes, `rollback()` undoes it

Python Classes

Python Intro
Variables and Data Types
Data Structures
Control Flow
Functions
Modules
Comprehensions
Exception Handling
Python IO
Python Database Access
Python Classes
APPENDIX Type Hinting
APPENDIX Python Libraries

Object Orientation in Python
Example Class and Objects
Creating Classes
A First Attribute and Method
Example Creating Classes and Objects
Class Members
Creating and Using Objects
Accessing Attributes
Class Attributes and Instance Attributes
`__str__` and `__repr__`
Attributes Are Public
A Computed Attribute
When a Plain Attribute Is Not Enough
The Rule of Thumb
Internal Attributes

A Class Can Extend Another
Summary : Python Classes

SpiralTrain developer courses

Object Orientation in Python

- **Python is an object-oriented language :**
  - Every value in Python is an object, and every object has a class
  - Classes of your own are created with the `class` keyword
- **Class :**
  - A blueprint for a set of related objects
  - Defines the attributes and methods that every object of the class has
  - Attributes hold data, methods are functions; both are reached with dot notation
- **Object or instance :**
  - An individual object of a certain class, with its own identity
  - Its state is the current value of its attributes
  - State differs per object, behavior is shared by all instances
- **Inheritance :**
  - A class can extend another class, inheriting its attributes and methods

SpiralTrain developer courses

Example Class and Objects

- A class is a blueprint; each World of Warcraft character has its own state :

	Character Class	Object Character A	Object Character B
Instance Variables	Name	Jaina	Hogger
	Level	60	11
	Health	95	35
	Zone	Dalaran	Elwynn Forest
Instance Methods	Introduce Method		
	Take Damage Method		
	Heal Method		

SpiralTrain developer courses

Creating Classes

- **The `class` statement creates a new class :**
  - The name of the class follows the keyword `class`, then a colon
  - The indented block below it is the body of the class

```
class Character:
    """A World of Warcraft character, with nothing in it yet."""

    pass  # an empty block
```

- **The string on the first line of the body is the documentation string :**
  - It can be read as `Character.__doc__`
- **Calling the class creates an object :**

```
jaina = Character()
print(jaina)
```

```
<__main__.Character object at 0x0000024A00FD8D70>
```

A First Attribute and Method

- **One attribute, `name`, and one method, `introduce`:**

```
class Character:
    def __init__(self, name: str) -> None:
        self.name = name

    def introduce(self) -> None:
        print(f"I am {self.name}")

jaina = Character("Jaina")
thrall = Character("Thrall")
jaina.introduce()
thrall.introduce()
```

```
I am Jaina
I am Thrall
```

- **`Character("Jaina")` creates an object and runs `__init__` on it:**
  - `self` is the new object; `self.name = name` stores the name on it
- **`jaina.introduce()` runs `introduce` with `self` set to `jaina`**

Example Creating Classes and Objects

- The same pattern with more attributes and methods :

```
class Character:
    """A World of Warcraft character, with its own health."""
    def __init__(self, name: str, health: int) -> None:
        self.name = name
        self.health = health
    def take_damage(self, amount: int) -> None:
        self.health = max(self.health - amount, 0)
    def heal(self, amount: int) -> None:
        self.health = min(self.health + amount, 100)
    def show_status(self) -> None:
        print(f"{self.name} : {self.health} health")
```

- Call the class by its name and pass the arguments that `__init__` accepts :

```
jaina = Character("Jaina", 100)
```

Class Members

- `__init__()` :
  - The initialization method, often called the constructor
  - Called by Python when you create a new instance of this class
  - Sets the instance attributes, here `self.name` and `self.health`
- **Instance methods :**
  - Declared like normal functions, but the first parameter of each method is `self`
  - You do not pass `self` when you call a method, since Python adds it
- `self` **is a reference to the instance :**
  - Instance attributes are read and written as `self.health`
  - `jaina.take_damage(10)` changes the health of `jaina` and of no other character
- **Python frees objects for you when nothing refers to them any more :**
  - `__del__` exists and you rarely need it

Creating and Using Objects

- **Creating objects :**

```
jaina = Character("Jaina", 100)
hogger = Character("Hogger", 60)
```

- **Access the object's methods and attributes using the dot operator :**

```
hogger.take_damage(25)
jaina.take_damage(10)
jaina.heal(5)

jaina.show_status()
hogger.show_status()
```

- **Result :**

```
Jaina : 95 health
Hogger : 35 health
```

- **Each object keeps its own `health` ; the methods are shared**

Each Object Has Its Own Health

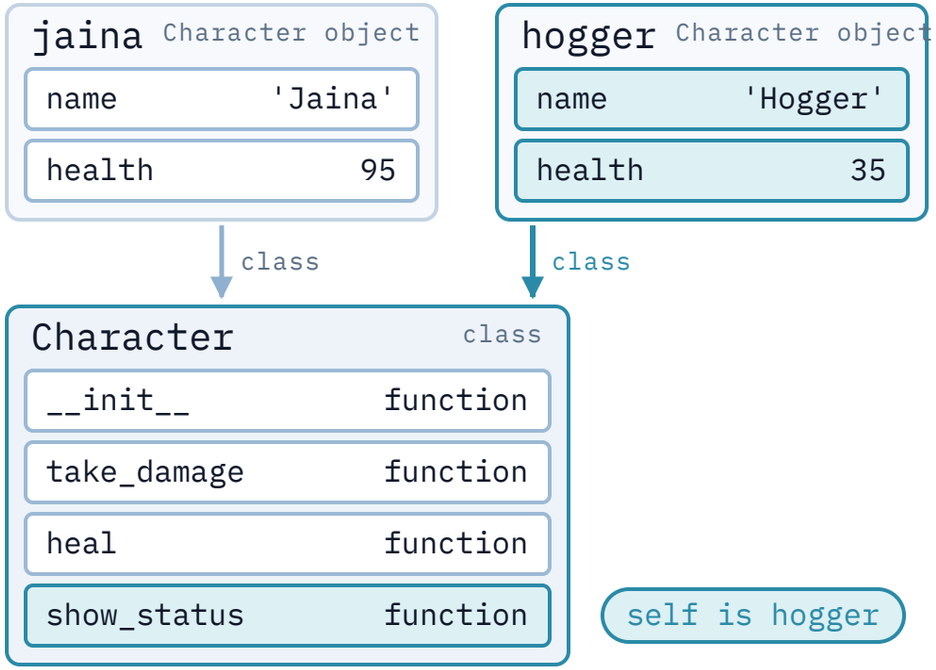
```
jaina = Character("Jaina", 100)
hogger = Character("Hogger", 60)

hogger.take_damage(25)
jaina.take_damage(10)
jaina.heal(5)

jaina.show_status()
hogger.show_status()
```

OUTPUT

```
Jaina : 95 health
Hogger : 35 health
```



The same method on `hogger` prints Hogger's own values

SpiralTrain developer courses

Accessing Attributes

- **In Python you can add, remove, or modify attributes at any time :**

```
jaina.mount = "Gryphon"  # Add a 'mount' attribute
jaina.mount = "Swift Gryphon"  # Modify 'mount' attribute
del jaina.mount  # Delete 'mount' attribute
```

- **Four functions do the same with an attribute name held in a string :**
  - Useful when the name is only known while the program runs

```
attr = "mount"  # an attribute name held in a variable
print(hasattr(jaina, attr))  # True if the attribute exists
setattr(jaina, attr, "Swift Gryphon")  # Set or create the attribute
print(getattr(jaina, attr))  # Returns the value of the attribute
delattr(jaina, attr)  # Delete the attribute
print(getattr(jaina, attr, "no mount"))  # The default, because the attribute is gone
```

```
False
Swift Gryphon
no mount
```

Class Attributes and Instance Attributes

```python
class Character:
    max_level = 60  # class attribute, one value shared by every character
    def __init__(self, name: str) -> None:
        self.name = name  # instance attribute, one value per character

jaina = Character("Jaina")
thrall = Character("Thrall")
print(jaina.name, thrall.name)  # Jaina Thrall
print(jaina.max_level, thrall.max_level)  # 60 60
Character.max_level = 70  # an expansion raises the cap for everyone
print(jaina.max_level, thrall.max_level)  # 70 70
jaina.max_level = 80  # creates an instance attribute on jaina only
print(jaina.max_level, thrall.max_level, Character.max_level)  # 80 70 70
```

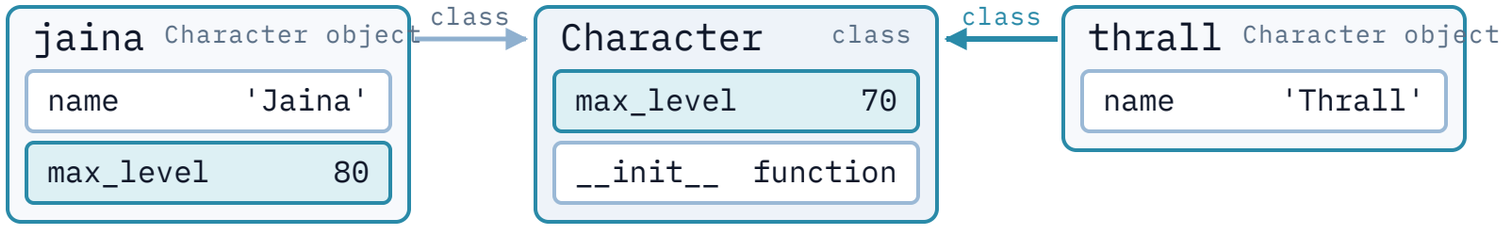
- **A class attribute is assigned in the class body; an instance attribute on `self`**
- **Reading `jaina.max_level` looks on the object first, then on the class**
- **Assigning `jaina.max_level` writes to the object; change shared data on the class**
- **Methods that need no instance exist as well, `@staticmethod` and `@classmethod` :**
  - They are covered in the Advanced Python course

demo04 · classattributes

Object First, Then Class

```python
class Character:
    max_level = 60  # class attribute, one value shared by every character
    def __init__(self, name: str) -> None:
        self.name = name  # instance attribute, one value per character

jaina = Character("Jaina")
thrall = Character("Thrall")
print(jaina.name, thrall.name)  # Jaina Thrall
print(jaina.max_level, thrall.max_level)  # 60 60
Character.max_level = 70  # an expansion raises the cap for everyone
print(jaina.max_level, thrall.max_level)  # 70 70
jaina.max_level = 80  # creates an instance attribute on jaina only
print(jaina.max_level, thrall.max_level, Character.max_level)  # 80 70 70
```



OUTPUT

```
Jaina Thrall
60 60
70 70
80 70 70
```

`jaina.max_level` is found on `jaina` first; `thrall.max_level` and `Character.max_level` still read the class

`__str__` and `__repr__`

```
class Character:
    def __init__(self, name: str, health: int) -> None:
        self.name = name
        self.health = health
    def __repr__(self) -> str:
        return f"Character({self.name!r}, {self.health!r})"
    def __str__(self) -> str:
        return f"{self.name} ({self.health} health)"

jaina = Character("Jaina", 100)
print(jaina)  # str(), for people
print(repr(jaina))  # repr(), for developers
print([jaina])  # a list shows the repr() of its items
```

```
Jaina (100 health)
Character('Jaina', 100)
[Character('Jaina', 100)]
```

- **`__str__` is used by `print()`, `str()` and f-strings**
- **`__repr__` is used by the interactive prompt, by containers and by `!r`:**
  - Make it look like the call that creates the object; `str()` falls back to it

Attributes Are Public

- **An attribute set in `__init__` is public, and that is fine :**
  - Any code that has the object can read it and assign to it
  - No methods are written to get or set it

```
class Character:
    def __init__(self, name: str, health: int) -> None:
        self.name = name
        self.health = health

jaina = Character("Jaina", 100)
print(jaina.health)  # read it
jaina.health = 80  # write it
print(jaina.health)
```

```
100
80
```

A Computed Attribute

- **Whether a character is alive depends on its health :**
  - As a method, `def is_alive(self)`, it is called as `jaina.is_alive()`
  - With `@property` above the `def`, it is read as `jaina.is_alive`, like an attribute

```
class Character:
    def __init__(self, name: str, health: int) -> None:
        self.name = name
        self.health = health
    @property
    def is_alive(self) -> bool:
        return self.health > 0

jaina = Character("Jaina", 30)
print(jaina.is_alive)  # no parentheses
jaina.health = 0
print(jaina.is_alive)  # always current
jaina.is_alive = True  # read-only : there is no setter
```

```
True
False
AttributeError: property 'is_alive' of 'Character' object has no setter
```

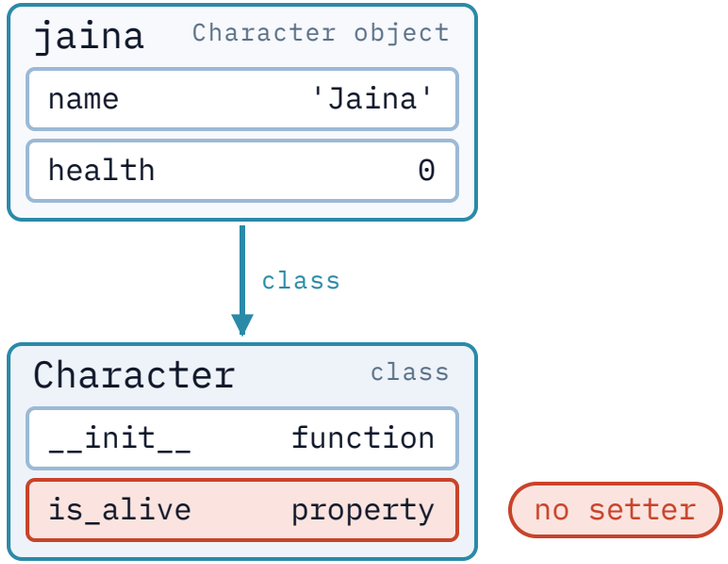
Computed on Every Read

```python
class Character:
    def __init__(self, name: str, health: int) -> None:
        self.name = name
        self.health = health
    @property
    def is_alive(self) -> bool:
        return self.health > 0

jaina = Character("Jaina", 30)
print(jaina.is_alive)  # no parentheses
jaina.health = 0
print(jaina.is_alive)  # always current
jaina.is_alive = True  # read-only : there is no setter
```

OUTPUT

```
True
False
AttributeError: property 'is_alive' of 'Character' object has no setter
```



The property has no setter, so assigning `jaina.is_alive` raises `AttributeError`

demo07 · computedproperty

When a Plain Attribute Is Not Enough

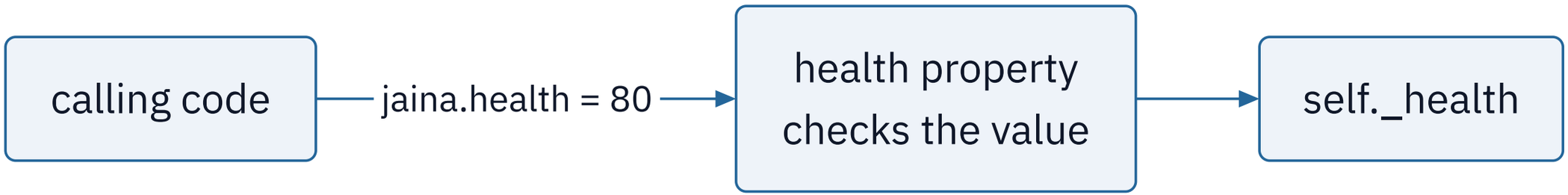
- The `Character` so far accepts any health, `jaina.health = -20` included
- The fix makes `health` a property whose setter checks the value :

```
class Character:
    def __init__(self, name: str, health: int) -> None:
        self.name = name
        self.health = health  # goes through the setter below
    @property
    def health(self) -> int:
        return self._health
    @health.setter
    def health(self, value: int) -> None:
        if not 0 <= value <= 100:
            raise ValueError(f"health must be between 0 and 100, got {value}")
        self._health = value
```

- The value is stored in `self._health`, the property's own storage
- The code that uses the class does not change :
  - `jaina.health = 0` still works, and `jaina.health = -20` now raises `ValueError`

The Rule of Thumb

- **Start with a plain attribute :**
  - Do not write getter and setter methods in advance
- **Turn it into a property the day it needs a rule :**
  - A check on the value, or a value computed from other attributes
  - The calling code keeps writing `jaina.health`, so nothing else changes
- **The property keeps its value in an attribute with a leading underscore :**
  - `_health` is internal to the class, by convention



SpiralTrain developer courses

Internal Attributes

- **Python has no keyword that makes an attribute private :**
  - A leading underscore, `_gold`, means internal, by convention
  - Code outside the class can reach it, and is expected not to
- **Two leading underscores, `__auth_code`, make Python rename the attribute to `_Character__auth_code`**

```
class Character:
    def __init__(self, name: str) -> None:
        self.name = name  # public
        self._gold = 0  # internal, by convention
        self.__auth_code = "7f3a"  # name mangled to _Character__auth_code

jaina = Character("Jaina")
print(jaina.name)
print(jaina._gold)  # works, but the underscore says: not for you
print(jaina._Character__auth_code)  # the mangled name still works
```

```
Jaina
0
7f3a
```

A Class Can Extend Another

```python
class Character:
    def __init__(self, name: str, health: int) -> None:
        self.name = name
        self.health = health
    def describe(self) -> str:
        return f"{self.name} has {self.health} health"

class Mage(Character):
    def __init__(self, name: str, health: int, spells: list[str]) -> None:
        super().__init__(name, health)  # the Character part
        self.spells = spells  # what a Mage adds
    def describe(self) -> str:  # replaces Character.describe
        return f"{super().describe()} and knows {len(self.spells)} spells"
```

- `Mage` **inherits every method of** `Character`**, adds to it and may replace methods**
- `super().__init__(...)` **runs the** `Character` **initialization, so** `name` **and** `health` **are set**
- `Mage("Jaina", 70, ["Fireball", "Frostbolt", "Blizzard"]).describe()` **returns :**
  - `Jaina has 70 health and knows 3 spells`
- **Inheritance in depth is covered in the Advanced Python course**

SpiralTrain developer courses

Summary : Python Classes

- **Classes are blueprints for a set of related objects :**
  - Define both attributes and methods and are created with the `class` keyword
- **Objects are instances of classes :**
  - Each object has its own state and identity, instance methods have a `self` parameter
- **The initialization method is named `__init__()` :**
  - Called by Python when the class is called to create an object
- **Instance attributes belong to one object, class attributes are shared by all instances**
- **`__str__` and `__repr__` decide how an object is printed**
- **Attributes are public; a property adds a rule without changing the calling code :**
  - Start with a plain attribute, turn it into a property the day it needs a rule
- **A leading underscore marks an attribute as internal, by convention**
- **A class can extend another, and calls `super().__init__()` to set up the parent part**

Type Hinting

Python Intro
Variables and Data Types
Data Structures
Control Flow
Functions
Modules
Comprehensions
Exception Handling
Python IO
Python Database Access
Python Classes
APPENDIX Type Hinting
APPENDIX Python Libraries

Type Hinting Recap
Basic Type Syntax
Typing Collections
Optional Values
Hints Are Not Enforced
Checking with mypy
Closed Sets with Literal
Summary : Type Hinting

Type Hinting Recap

- **Type hints have appeared alongside the examples throughout the course :**
  - Values such as `seconds: int = 222`
  - Collections such as `list[str]` and `dict[str, int]`
  - Functions such as `format_duration(seconds: int) -> str`
  - Constructors and methods in Classes
- **An annotation records the intended type :**
  - Editors use it for completion and navigation
  - A checker finds code that contradicts it
  - Python does not enforce it while the program runs
- **This appendix adds optional values, closed sets and a checker**

Basic Type Syntax

- A colon introduces a variable or parameter type; an arrow introduces the result :

```
def format_duration(seconds: int) -> str:
    minutes = seconds // 60
    remainder = seconds % 60
    return f"{minutes}:{remainder:02d}"
```

- `None` describes a function without a useful returned result :

```
def print_playlist(name: str, tracks: list[str]) -> None:
    print(f"Playlist: {name}")
    for number, title in enumerate(tracks, start=1):
        print(f"{number}. {title}")
```

- Variable annotations use the same types : `seconds: int = 222`

Typing Collections

- Collections state the types of their contents :

```
tracks: list[str] = ["Imagine", "Yesterday", "Respect"]
favourites: set[str] = {"Imagine", "Yesterday"}
durations: dict[str, int] = {"Imagine": 183, "Yesterday": 125}
```

- A tuple describes each position; an ellipsis allows any length of one type :

```
photo_size: tuple[int, int] = (1920, 1080)
track: tuple[str, int] = ("Imagine", 183)
album_lengths: tuple[int, ...] = (183, 125, 149)
```

- The dictionary type gives the key type first, then the value type

Optional Values

- A lookup may return a duration or no value :

```
def find_duration(title: str, durations: dict[str, int]) -> int | None:
    return durations.get(title)
```

- Test for `None` before using the duration :

```
durations = {"Imagine": 183, "Yesterday": 125}
seconds = find_duration("Wonderwall", durations)
if seconds is None:
    print("Track not found")
else:
    print(f"Track length: {seconds} seconds")
print(parse_seconds("222"))
```

- `str | int` similarly accepts either a string or an integer
- A default value is separate from the type; `| None` does not make an argument optional to pass

A Lookup That May Find Nothing

```python
def find_duration(title: str, durations: dict[str, int]) -> int | None:
    return durations.get(title)


durations = {"Imagine": 183, "Yesterday": 125}
seconds = find_duration("Wonderwall", durations)
if seconds is None:
    print("Track not found")
else:
    print(f"Track length: {seconds} seconds")
```

durations `dict[str, int]`

'Imagine'	183
'Yesterday'	125

returns `int | None`

int

None

seconds → None

is None → True

OUTPUT

```
Track not found
```

The `None` branch reports the missing track; the `else` branch would receive an `int`

Hints Are Not Enforced

- A wrong type can reach the function without an immediate runtime error :

```
def repeat_track(title: str, count: int) -> list[str]:
    return [title] * count


print(repeat_track(222, 2))  # [222, 222], despite the str annotation.
```

- The result contains numbers even though the return type says `list[str]`
- Annotations do not convert or validate incoming values
- Annotations remain available through `repeat_track.__annotations__`
- A checker can report the mismatch before this program runs

demo04 · not_enforced

SpiralTrain developer courses

Checking with mypy

- `mypy` checks calls against the declared parameter types :

```
def repeat_track(title: str, count: int) -> list[str]:
    return [title] * count


queue: list[str] = repeat_track(222, 2)
```

```
uv add --dev mypy
uv run mypy 05_mypy_error.py
```

```
error: Argument 1 to "repeat_track" has incompatible type "int"; expected "str"
Found 1 error in 1 file (checked 1 source file)
```

demo05 · mypy_error

Closed Sets with Literal

- `Literal` **lets a checker restrict a value to specific options :**
  - This player implements normal playback and repeating the current track

```
from typing import Literal

type PlaybackMode = Literal["normal", "repeat"]


def next_index(index: int, mode: PlaybackMode) -> int:
    if mode == "repeat":
        return index
    return index + 1


print(next_index(0, "repeat"))
print(next_index(0, "shuffle"))  # Deliberate checker error.
```

- `mypy` **flags** `"shuffle"` **because it is outside the declared options**
- **Runtime validation still needs ordinary Python checks**

SpiralTrain developer courses

Summary : Type Hinting

- `uv` **manages the interpreter, the environment and the packages :**
  - `uv python install` , `uv init` , `uv add` , `uv sync` , `uv run`
  - `pyproject.toml` states what is wanted, `uv.lock` pins what was resolved
- `ruff` **reports style and correctness problems without running the code :**
  - `ruff check .` and `ruff check . --fix` , configured in `ruff.toml`
- **Type hints record in the signature what a function expects and returns :**
  - `age: int` for a variable, `def f(a: int) -> str` for a function
  - `list[int]` , `dict[str, int]` and `tuple[int, int]` for containers
  - `str | None` for a value that may be absent
  - `Literal["normal", "repeat"]` for a closed set of literal values
- **Annotations are ignored at runtime :**
  - Their value comes from the editor and from a checker such as `mypy`

Python Libraries

Python Intro
Variables and Data Types
Data Structures
Control Flow
Functions
Modules
Comprehensions
Exception Handling
Python IO
Python Database Access
Python Classes
APPENDIX Type Hinting
APPENDIX Python Libraries
Standard Library Overview
The datetime Module
Formatting and Parsing Dates
Date Arithmetic with timedelta
The json Module
Reading and Writing JSON Files
JSON and Python Types
Regular Expressions with re
Matching and Searching
Groups and Capturing
Substitution and Splitting
Command-Line Arguments with sys.argv
Testing with pytest
Reading a Test Failure
Exceptions and Parametrized Tests
Summary : Python Libraries

SpiralTrain developer courses

Standard Library Overview

- **Python ships with a large standard library :**
  - Available on every installation, with nothing to add and nothing to pin
  - Reach for it before reaching for a third party package
- **The modules this appendix covers :**
  - `datetime` , dates, times and the arithmetic between them
  - `json` , reading and writing the interchange format of the web
  - `re` , regular expressions for searching and rewriting text
  - `sys.argv` , inspecting command-line arguments passed to a script
- `pytest` **is third-party software for testing with plain assertions :**
  - Exercise 13 installs it with `uv add --dev pytest`
- **Others worth knowing about :**
  - `pathlib` for filesystem paths, `csv` for tabular files
  - `collections` for counters, deques and default dictionaries
  - `itertools` for combinations, grouping and infinite sequences
  - `argparse` for flags, options and generated help messages
- **Full reference at** `https://docs.python.org/3/library/`

The datetime Module

- **Three types cover almost every need :**
  - `date` is a calendar day with no time on it
  - `time` is a time of day with no date on it
  - `datetime` is both together

```
from datetime import date, datetime, time

photo_day = date(2026, 8, 15)
midday = time(12, 30, 0)
taken_at = datetime(2026, 8, 15, 12, 30, 0)

print(taken_at.year, taken_at.month, taken_at.day)
print(taken_at.weekday())  # Monday is 0, so this prints 5
```

- **The current moment comes from the class itself :**
  - `date.today()` and `datetime.now()`

Formatting and Parsing Dates

- `strftime` formats a datetime as text, `strptime` reads text back :
  - The `f` is for format, the `p` is for parse

```
taken_at.strftime("%d-%m-%Y %H:%M")  # 15-08-2026 12:30
taken_at.strftime("%A %d %B %Y")     # Saturday 15 August 2026

datetime.strptime("24-12-2026 18:00", "%d-%m-%Y %H:%M")
```

- Common directives :
  - `%Y` four digit year, `%m` month, `%d` day
  - `%H` hour, `%M` minute, `%S` second
  - `%A` weekday name, `%B` month name
- ISO 8601 needs no format string in either direction :

```
taken_at.isoformat()  # 2026-08-15T12:30:00
datetime.fromisoformat("2026-12-24T18:00:00")
```

demo02 · datetime_format

Date Arithmetic with timedelta

- **Subtracting two dates gives a `timedelta`, a duration :**

```
from datetime import date, timedelta

start = date(2026, 8, 15)
end = date(2026, 12, 24)
span = end - start
print(span.days)  # 131
```

- **Adding a `timedelta` to a date gives another date :**

```
start + timedelta(weeks=1)   # 2026-08-22
start + timedelta(days=100)  # 2026-11-23
start - timedelta(days=1)    # 2026-08-14
```

- **A `timedelta` is built from any combination of units :**
  - `days` , `hours` , `minutes` , `seconds` , `weeks`
  - It normalises itself, so 25 hours becomes one day and one hour

The json Module

- **JSON stores structured text exchanged by programs :**
  - `dumps` converts a Python value to JSON text; `loads` reads it back

```
import json

playlist = {"name": "Night drive", "tracks": ["Imagine", "Yesterday"], "cover": None}
text = json.dumps(playlist, indent=2, sort_keys=True)
restored = json.loads(text)
```

```
{
  "cover": null,
  "name": "Night drive",
  "tracks": [
    "Imagine",
    "Yesterday"
  ]
}
```

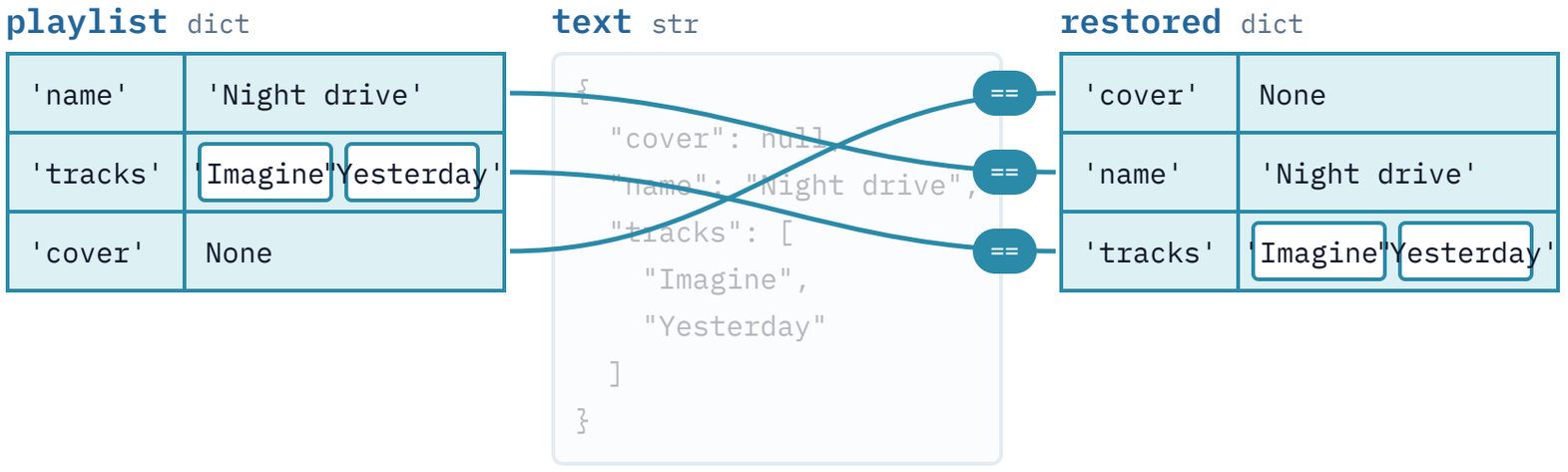
- **`indent` makes the output easier to read; `sort_keys` orders dictionary keys**

demo04 · json_strings

A Playlist Round Trip

```python
import json

playlist = {"name": "Night drive", "tracks": ["Imagine", "Yesterday"], "cover": None}
text = json.dumps(playlist, indent=2, sort_keys=True)
restored = json.loads(text)
print(text)
print(restored == playlist)
```



OUTPUT

```
{
  "cover": null,
  "name": "Night drive",
  "tracks": [
    "Imagine",
    "Yesterday"
  ]
}
True
```

`==` pairs the entries by key, whatever their order, and compares the lists item by item: `True`

SpiralTrain developer courses

Reading and Writing JSON Files

- `dump` and `load` work with file objects :
  - The names are the string versions without the trailing `s`

```
import json
from pathlib import Path

playlist = {"name": "Night drive", "tracks": ["Imagine", "Yesterday"], "cover": None}
path = Path("playlist.json")

with path.open("w", encoding="utf-8") as file:
    json.dump(playlist, file, indent=2)

with path.open(encoding="utf-8") as file:
    restored = json.load(file)
print(restored["tracks"])
```

- Explicit UTF-8 keeps playlist files consistent across machines

SpiralTrain developer courses

JSON and Python Types

- **The mapping between the two type systems is not quite one to one :**

Python	JSON
`dict`	object
`list` , `tuple`	array
`str`	string
`int` , `float`	number
`True` / `False`	`true` / `false`
`None`	`null`

- **Note what this costs on a round trip :**
  - A `tuple` comes back as a `list`
  - Dictionary keys come back as strings, whatever they went in as
  - `set` , `datetime` and your own classes are not serialisable at all
- **For those, convert to a supported type first, or pass `default=`**

SpiralTrain developer courses

Regular Expressions with re

- **A regular expression is a pattern describing a set of strings :**
  - Written as a raw string, `r"..."`, so backslashes reach the engine intact
- **The character classes that do most of the work :**
  - `\d` digit, `\w` word character, `\s` whitespace
  - `.` any character, `[abc]` one of these, `[^abc]` none of these
- **The repetition operators :**
  - `*` zero or more, `+` one or more, `?` zero or one
  - `{3}` exactly three, `{2,4}` between two and four
- **Anchors tie a pattern to a position :**
  - `^` start of the string, `$` end of the string, `\b` word boundary

```
r"\d{4}-\d{2}-\d{2}"   # a date like 2026-08-15
r"^IMG_\d{4}\.jpg$"   # a photo filename like IMG_0421.jpg
```

SpiralTrain developer courses

Matching and Searching

- **Regex functions differ in where and how much they match :**
  - `match` starts at position zero; `search` scans the whole string
  - `findall` returns all matches; `fullmatch` requires the whole string

```
import re

text = "Playing Imagine (183s), Yesterday (125s)"
print(re.match(r"Playing", text) is not None)  # True
print(re.match(r"Yesterday", text) is not None)  # False
print(re.search(r"Yesterday", text) is not None)  # True
print(re.findall(r"\d+", text))  # ['183', '125']
print(re.fullmatch(r"IMG_\d{4}\.jpg", "IMG_0421.jpg") is not None)  # True
print(re.fullmatch(r"IMG_\d{4}\.jpg", "notes.txt") is not None)  # False
```

- **`match`, `search` and `fullmatch` return a match object or `None`**
- **Flags include `re.IGNORECASE`, `re.MULTILINE` and `re.DOTALL`**

demo06 · regex_search

SpiralTrain developer courses

Groups and Capturing

- Parentheses capture useful parts of a player log entry :

```
import re

line = "12:30 PLAY Imagine"
match = re.search(r"(\d{2}):(\d{2}) (\w+) (.*)", line)
if match is not None:
    print(match.group(0))  # 12:30 PLAY Imagine
    print(match.groups())  # ('12', '30', 'PLAY', 'Imagine')
```

- Named groups identify captured values without counting positions :

```
named = re.search(r"(?P<action>PLAY|SKIP) (?P<title>.*)", line)
if named is not None:
    print(named.group("action"))  # PLAY
    print(named.group("title"))  # Imagine
```

Substitution and Splitting

- `sub` replaces matching text :
  - A backreference such as `\1` reuses a captured group
- `re.split` uses a pattern to separate items :

```
import re

text = "IMG_0421.JPG; IMG_0422.JPG"
print(re.sub(r"IMG_\d{4}\.JPG", "[photo]", text))
print(re.sub(r"IMG_(\d{4})\.JPG", r"photo_\1.jpg", text))
print(re.split(r";\s*", text))
```

```
[photo]; [photo]
photo_0421.jpg; photo_0422.jpg
['IMG_0421.JPG', 'IMG_0422.JPG']
```

- The identifier stays available when a photo filename is reformatted

Command-Line Arguments with sys.argv

- `sys.argv` **contains the arguments passed to a script :**
  - Position `0` is the script path; later positions contain its arguments

```
import sys

print(f"Script: {sys.argv[0]}")
playlist_file = sys.argv[1] if len(sys.argv) > 1 else "night-drive.json"
print(f"Playlist file: {playlist_file}")
```

```
uv run python 09_sys_argv.py warm-up.json
```

- **Arguments arrive as strings; convert numeric values explicitly**
- `argparse` **adds option flags, defaults and generated help**

SpiralTrain developer courses

Testing with pytest

- **Tests can check the duration formatter already used by the player :**
  - Each test function starts with `test_`
  - A plain `assert` states the expected result

```python
def format_duration(seconds: int) -> str:
    minutes = seconds // 60
    remainder = seconds % 60
    return f"{minutes}:{remainder:02d}"


def test_minutes_and_seconds() -> None:
    assert format_duration(222) == "3:42"


def test_exact_minute() -> None:
    assert format_duration(60) == "1:00"
```

```
uv add --dev pytest
uv run pytest 10_pytest_basic.py
```

Reading a Test Failure

- A missing zero-padding rule is a plausible display bug :

```
def format_duration(seconds: int) -> str:
    minutes = seconds // 60
    remainder = seconds % 60
    return f"{minutes}:{remainder}"


def test_exact_minute() -> None:
    assert format_duration(60) == "1:00"
```

- Running `uv run pytest 11_pytest_failure.py` exposes the difference :

```
E       AssertionError: assert '1:0' == '1:00'
```

- Restore `:02d` in the f-string to satisfy the test

SpiralTrain developer courses

Exceptions and Parametrized Tests

- `pytest.raises` checks a failure that the parser must report
- `parametrize` repeats one test with several inputs :

```
def test_invalid_duration() -> None:
    with pytest.raises(ValueError):
        parse_duration("3:99")


@pytest.mark.parametrize(
    ("seconds", "expected"),
    [(0, "0:00"), (59, "0:59"), (60, "1:00"), (3600, "60:00")],
)
def test_duration_display(seconds: int, expected: str) -> None:
    assert format_duration(seconds) == expected
```

- The demo includes the familiar formatter and duration parser
- Fixtures, mocking and testing in depth belong to Advanced Python

SpiralTrain developer courses

Summary : Python Libraries

- `datetime` **covers dates, times and durations :**
  - `date` , `time` , `datetime` and `timedelta`
  - `strftime` formats and `strptime` parses using a pattern
  - `isoformat()` formats; `fromisoformat()` parses, without a format pattern
- `json` **converts between Python objects and JSON text :**
  - `dumps` and `loads` for strings, `dump` and `load` for files
  - A tuple returns as a list, and keys return as strings
- `re` **searches and rewrites text with patterns :**
  - `match` at the start, `search` anywhere, `findall` for every hit
  - Parentheses capture, and `sub` replaces using what was captured
- `sys.argv` **exposes command-line arguments as a list of strings :**
  - `argparse` is the next step for options, flags and automatic help
- `pytest` **runs checks over the code automatically :**
  - Test functions named `test_` and plain `assert` statements
  - `pytest.raises` for exceptions and `@pytest.mark.parametrize` for tables